Imperial College London

Imperial College London

Department of Computing

Finding and Understanding Bugs in Zero-Knowledge Proof Compilers

Author:
Jakub Cabała

Supervisors:
Prof. Alastair Donaldson
Dr. Stefanos Chaliasos

Second Marker:
Prof. Cristian Cadar

MEng Individual Project

Final Report

June 10, 2026

Abstract

Zero-knowledge proof (ZKP) compilers translate high-level programs into arithmetised constraint systems used to generate cryptographic proofs. Bugs in this translation process can introduce semantic discrepancies between source programs and generated constraints, potentially compromising correctness. While existing ZKP compiler fuzzers have successfully identified crashes and implementation defects, little work has focused on testing the logical correctness of generated constraints.

This project investigates SMT-based semantic testing of ZKP compilers. I first extend the existing CIRCUZZ fuzzer with improved support for generating programs with constraints. I then design and implement an SMT-based testing framework that generates structured seed programs from SMT formulas, compiles them into ZKP programs, and analyses the resulting constraint systems using SMT solvers and constrainedness-analysis tools as testing oracles.

I evaluate the framework using compiler coverage measurements, artificial bug-injection experiments, and real-world fuzzing campaigns. The results show that structured SMT-generated programs effectively exercise ZKP compiler optimisation pipelines and detect injected semantic faults noticeably faster than the original CIRCUZZ workflow.

In the real-world fuzzing campaigns, the framework did not uncover verified logical constraint-generation bugs in the evaluated compilers, but the broader project nevertheless identified 6 previously unknown issues in ZKP compiler pipelines, as well as 2 issues in SMT solver infrastructure.

Overall, the results demonstrate that SMT-based semantic testing is a viable approach for analysing compiler correctness. They also suggest that SMT-based reasoning is best used alongside complementary testing oracles rather than in isolation. Such combinations can broaden the range of detectable bugs, improve testing effectiveness, and reduce the risk of over-specialising towards a particular class of bugs.

Acknowledgements

I would like to thank my supervisor, Prof. Alastair Donaldson, for his guidance and support throughout this project. His expertise in compiler testing and fuzzing, together with the many papers, suggestions, and ideas he shared, were invaluable to the development of this work. I am particularly grateful for his genuine interest in the project and for the regular meetings and discussions that helped shape its direction.

I would also like to thank Dr. Stefanos Chaliasos for his support on the zero-knowledge proof side of the project. His enthusiasm for the subject, excellent intuition, and willingness to discuss and refine new ideas were invaluable throughout the year. I greatly appreciated his responsiveness, insightful feedback, and the many productive discussions that helped shape the direction of this work.

Finally, I would like to thank Prof. Cristian Cadar for serving as my second marker and for the helpful feedback provided during the interim report meeting.

Contents

Chapter 1

Introduction

1.1 Motivation and Objectives

Zero-knowledge proofs (ZKPs) are cryptographic protocols that enable the verification of statements about private data. Through such a protocol, a prover P can convince a verifier V that they know some secret data satisfying a given predicate, while revealing no additional information about that data [1]. ZKPs have important applications in blockchain systems [2] and substantial economic impact. For example, Zcash (ZEC), one of the leading privacy cryptocurrencies based on ZKPs, currently represents roughly $8 billion USD in market capitalization [3].

In practice, many ZKPs are expressed as programs written in domain-specific languages (DSLs) such as Circom [4], Noir [5], or Gnark [6]. These programs are then translated into low-level constraint systems and witness generators used by the proof system. Correctly performing this translation is critical, as compiler bugs may allow invalid proofs to be accepted, leading to severe security and financial consequences [7].

This project investigates automated testing techniques for ZKP compilers, with a particular focus on detecting logical bugs in generated constraint systems. I first study existing tools, including MTZK [8] and Circuzz [7], and analyse their testing methodologies and oracle designs. This investigation suggests that, while existing approaches are effective at detecting crashes and implementation issues, they are less suited to finding logical bugs in generated constraints because of the behaviours their oracles check.

Motivated by this limitation, I design and implement a new testing framework inspired by SMT-solver testing techniques [9]. The framework generates structured seed programs from SMT formulas, compiles them through ZKP compiler pipelines, and directly analyses the resulting constraints. This allows the framework to target semantic inconsistencies between source programs and generated constraints.

The main objectives of the project are therefore to improve existing ZKP compiler testing infrastructure, design SMT-based generators and oracles for constraint-level testing, evaluate their effectiveness and limitations, and investigate optimisations for improving scalability and bug-finding performance.

1.2 Contributions

This project makes the following key contributions:

- I extend the Circuzz framework by improving its Circom backend so that generated programs exercise constraint generation rather than only witness-generation checks, and by adding support for additional ZKP languages.
- I design and implement an SMT-based generator that produces structured seed programs together with satisfying witness values for testing ZKP compiler constraint generation.
- I design and implement an oracle that directly inspects generated constraints to detect semantic

correctness bugs and is able to efficiently target soundness bugs.

- I generate a collection of finite-field SMT benchmarks derived from ZKP circuits and by other means, providing additional evaluation workloads for SMT solvers with finite-field theories - a domain where very few public benchmarks currently exist.
- I release the implementation and experimental infrastructure developed during this project as open-source software.[1]

Additionally, during the course of this project, I identified 6 bugs in ZKP compiler pipelines and 2 bugs in the finite-field solving backend of the CVC5 SMT solver - one of which was novel and another was a known issue.

1.3 Structure

The remainder of this report is structured as follows:

- Chapter 2 provides the necessary background material, including the theory, terminology, and overview of tools used throughout the project.
- Chapter 3 presents an overview of the existing testing frameworks together with the experiments, improvements, and conclusions developed early during the project.
- Chapter 4 presents the SMT-based constraint-generation testing framework developed in this project, together with its late-stage extension to a higher-level DSL.
- Chapter 5 presents the evaluation of the proposed approach, together with additional experiments and a discussion of the bugs identified during the project.
- Chapter 6 summarises the main conclusions of the project and discusses possible directions for future work.

[1] `https://github.com/jCabala/zkp-compiler-testing`

Chapter 2

Background

In this chapter, I introduce ZKPs and describe the role and structure of ZKP compilers, together with several important properties of circuits. I then survey the DSLs and frameworks whose compilers are the focus of this project. Subsequently, I provide an overview of the software testing techniques employed throughout the work. Finally, I review relevant background on SMT solvers, which provides the basis for the testing techniques developed in Chapter 4.

2.1 Zero-Knowledge Proofs

ZKPs are cryptographic protocols that allow a prover P to convince a verifier V of the validity of a statement without revealing any information beyond the fact that the statement is true [1]. Modern ZKP systems are predominantly used to prove statements about program execution. Rather than proving a simple mathematical claim, the prover demonstrates that a program was executed correctly on some combination of private and public inputs and outputs. This paradigm is the basis for many real-world applications, including privacy-preserving cryptocurrencies, rollups, confidential smart contracts and zero-knowledge virtual machines [2]. Any ZKP system must satisfy three fundamental properties:

- **Completeness**: The verifier should accept valid proofs.
- **Soundness**: The verifier cannot be convinced of the validity of an invalid statement (except with negligible probability).
- **Zero-knowledge**: The verifier learns nothing beyond the truth of the statement.

2.1.1 Algebraic Constraint Systems

In practice, to generate a ZKP, the computation must be encoded as an algebraic constraint system over a finite field, which is essentially a set of polynomial equations.

The most widely used representation is the *Rank-1 Constraint System* (R1CS) [10]. An R1CS consists of constraints of the form

$$\langle a_i, z\rangle \cdot \langle b_i, z\rangle = \langle c_i, z\rangle,$$

where z is the vector of variables containing all inputs, outputs and intermediate variables, a_i, b_i, c_i are fixed coefficient vectors over a finite field and $\langle \cdot, \cdot \rangle$ represents a dot product. In short, each constraint enforces that the product of two linear combinations of variables equals a third linear combination.

Example (R1CS encoding). Consider a computation that takes a private input x and produces a public output $y = x^2 + 3$. Let the variable vector be

$$z = (1, x, t, y),$$

where t is an intermediate variable representing x^2. This computation can be expressed using the following R1CS constraints:

$$x \cdot x = t,$$

$$t + 3 = y.$$

Together, these constraints enforce a correct derivation of output from the input.

R1CS is not the only constraint system used in practice. More recent proof systems often use PLONK-style constraints, which describe computation as a fixed arithmetic circuit rather than as a list of individual equations. The circuit is made of arithmetic gates connected by wiring constraints, and may also include lookup tables [11]. Another formulation used in practice is the Algebraic Intermediate Representation (AIR), which models computation as a sequence of state transitions over time. Instead of constraining individual wires or gates, AIR specifies algebraic constraints that relate consecutive rows of an execution trace, capturing how the machine state evolves step by step [12].

2.1.2 Proof Systems

Once an algebraic constraint system ϕ has been defined, a zero-knowledge proof system specifies how statements about the satisfiability of ϕ are proven and verified. A proof system is typically defined by three algorithms:

$$Setup(\phi) \rightarrow (pk, vk) \tag{2.1}$$

$$Prove(pk, x, w) \rightarrow \pi \tag{2.2}$$

$$Verify(vk, x, \pi) \rightarrow \{0, 1\}. \tag{2.3}$$

The *Setup* algorithm derives system-specific parameters, producing a proving key pk and a verification key vk. The *Prove* algorithm allows a prover, given pk, public inputs x, and a private witness w satisfying $\phi(x, w) = 1$, to construct a proof π proving the existence of such a witness. The *Verify* algorithm checks, using vk and the public inputs x, that the proof π is valid. Several proof systems are used in practice. *Groth16* [10] is an R1CS-based system with very small proofs and fast verification, but requires a circuit-specific trusted setup. *PLONK* [11] and *Halo2* [13] work over a fixed circuit structure and support a universal or recursive setup reusable across circuits. *STARKs* [12] avoid trusted setup entirely and rely on hash-based cryptography, at the cost of larger proofs.

Most of the practical systems, such as Groth16, PLONK and Halo2 are *Succinct Non-interactive Arguments of Knowledge* (SNARKs): proofs (not necessarily prove generation time) are short and the verification is efficient. These properties make SNARKs useful in practice, as they allow complex computations to be verified with minimal computational cost for the verifier [14].

2.1.3 The ZKP Compiler Pipeline

In practice, constraint systems are not written by hand but are generated automatically from high-level programs by a *ZKP compiler*. Given a program C, the compiler usually produces:

1. An algebraic constraint system ϕ over a finite field.
2. A witness generation function $WG(p, s) \rightarrow (x, w)$, which computes a satisfying assignment to ϕ from public inputs p and secret inputs s.

The compilation process typically involves lowering the high-level control flow and data types into arithmetic circuits, introducing intermediate variables, and simplifying the constraints. Figure 2.1 illustrates how the different components like constraint system, compiler and witness generator usually interact.

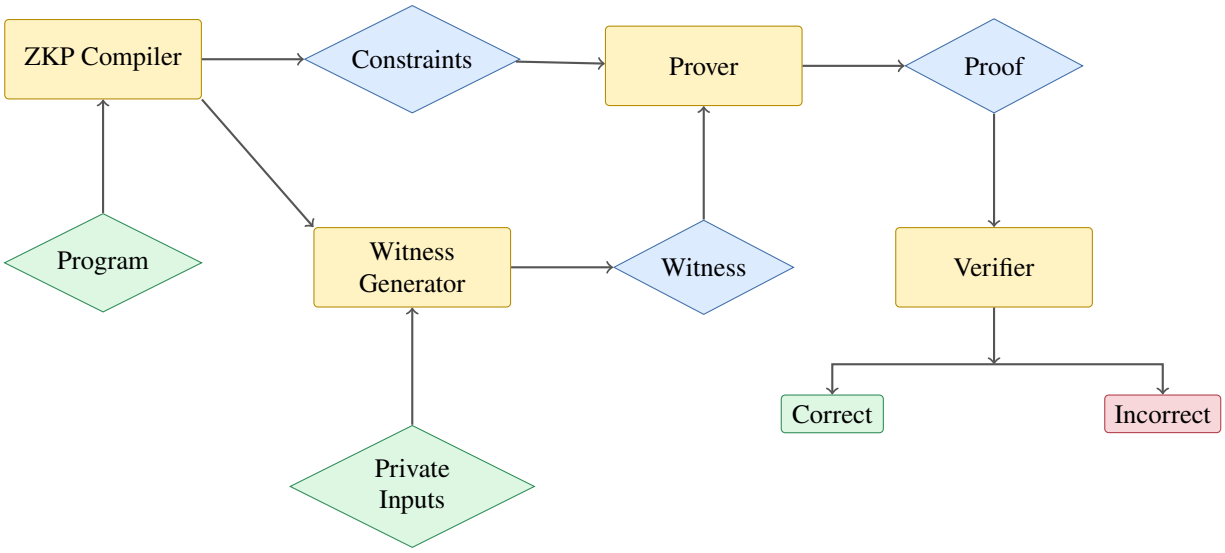

Figure 2.1: Visualisation of interactions between different components in a ZKP pipeline.

2.1.4 Equisatisfiability

A ZKP compiler is correct only if the constraint system it produces is *equisatisfiable* with the semantics of the original circuit. This means that the compiler preserves which inputs are considered valid: any computation that respects the circuit logic should yield a satisfying assignment to the generated constraints, and the existence of a satisfying assignment indicates that the circuit's conditions can be met for some witness values. Equisatisfiability ensures that the compiler neither rejects valid computations nor admits impossible ones, and is therefore a fundamental requirement for the soundness and completeness of the resulting proof system.

2.1.5 Types of Compiler Bugs

Compiler bugs in ZKP systems can be broadly categorised into *runtime failures* (crashes) and *logic bugs*. Runtime failures cause the compiler or generated prover to terminate unexpectedly during compilation or execution, whereas logic bugs violate the equisatisfiability between the original circuit and the generated constraint system. The two types of the logic bugs are:

- **Completeness bugs.**
  They occur when the compiler produces constraints that reject a valid witness.

  Example. Consider a circuit that computes the product of two private inputs, $y = a \cdot b$, over a field $\mathbb{F}$. A correct compilation would introduce a multiplicative constraint enforcing this relation. However, suppose the compiler erroneously generates the constraint

  $$y = a \cdot b + 1.$$

  Then any valid witness assignment satisfying $y = a \cdot b$ will fail to satisfy the generated constraint. As a result, the constraint system rejects a valid witness, violating completeness.
- **Soundness bugs.**
  They occur when the compiler generates constraints that admit invalid witnesses, allowing a prover to convince a verifier of false statements.

  Example. Consider the same circuit as above. Suppose the compiler instead emits the constraint

  $$y \cdot 0 = a \cdot b \cdot 0,$$

  which is trivially satisfied for any values of a, b, and y. As a result, a prover may supply an arbitrary value for y that does not correspond to the true product of the inputs, yet still produce a

valid proof. This violates soundness, as the verifier accepts a false statement about the circuit's intended semantics.

Soundness bugs are generally far more severe in practice than completeness bugs, and multiple real-world vulnerabilities have arisen from such issues [15]. A real-world example of this class of bug is the Circom-Pairing vulnerability, where a missing output check created a mismatch between the intended circuit semantics and the constraints actually enforced by the proof system. As a result, invalid public key values could satisfy the generated constraints, potentially enabling forged signatures [16, 17].

2.1.6 Constrainedness

A central cause of soundness bugs is *underconstrainedness*. A circuit is underconstrained when the constraints allow multiple possible outputs for the same inputs, instead of enforcing a single correct result.

Example (Underconstrained R1CS). Consider a circuit intended to enforce the relation $y = x^2$, where x is a private input and y is a public output. Let the variable vector be

$$z = (1, x, t, y),$$

where t is an intermediate variable. Suppose the constraint system consists of the single constraint

$$x \cdot x = t,$$

and omits the constraint equating the intermediate value t with the public output y. This R1CS is satisfiable for any public value y: for any choice of x, the prover can set $t = x^2$, while y remains unconstrained. As a result, multiple outputs y satisfy the constraint system for the same input x, violating the intended functional relation and breaking soundness.

Fix. The issue is resolved by adding the missing constraint

$$t - y = 0,$$

which enforces $y = t$ and restores functional determinism.

Underconstrained circuits can be viewed as exhibiting unintended non-determinism. A circuit is non-deterministic if there exist multiple valid witness assignments that satisfy the constraints while producing different outputs for the same inputs. In a correctly constrained circuit, the witness (and hence the output) should be uniquely determined by the inputs. Underconstrainedness violates this property, allowing multiple distinct behaviours to be accepted by the same constraint system. While some circuits may intentionally permit multiple valid outputs, this is uncommon in practice and, as a general guideline, non-determinism in a circuit should be treated with caution, as it often indicates missing constraints [18].

In contrast, a circuit is *overconstrained* if it admits fewer solutions than intended, rejecting valid executions of the computation. Overconstrained circuits affect completeness: there exist valid inputs and witnesses that satisfy the intended semantics but do not satisfy the constraint system. In practice, such issues are generally less common and considered less critical than underconstrained circuits, which directly compromise soundness [18].

2.1.7 Tools for Detecting Constrainedness Bugs

Because underconstrainedness directly compromises soundness, detecting such flaws is important in practical applications. A number of automated tools have been proposed to identify constraint-related bugs in ZK circuits.

One of those tools, that is based on symbolic execution is *Picus* [18, 19], which models the constraint system and checks for non-uniqueness of solutions. In particular, Picus determines whether multiple

distinct outputs can satisfy the same set of constraints for a fixed input, thereby identifying violations of functional correctness. This approach is precise and effective at uncovering subtle soundness bugs. However, it relies on SMT solvers (introduced in Section 2.4), and therefore quickly becomes computationally expensive and scales poorly for large or highly complex circuits.

More recently, another tool has been proposed - *ZKFUZZ* [20]. In contrast to Picus, ZKFUZZ explores concrete executions by generating inputs and mutating programs to produce diverse execution traces. This allows it to scale efficiently to large circuits and detect bugs quickly in practice. However, unlike Picus, ZKFUZZ does not provide completeness guarantees: failure to find a bug does not imply correctness. On the other hand, it is able to capture a broader class of issues, including both underconstrained and overconstrained circuits, whereas Picus is limited to detecting non-uniqueness in constraints [20].

2.2 Existing Domain-Specific Languages for ZKPs

Several domain-specific languages (DSLs) and frameworks for ZKPs have been developed, each making different trade-offs between abstraction level, and control over the generated constraint system. Below I briefly overview the languages and frameworks that are relevant for this work, focusing on what they are used for, the type of constraints they generate, and the proving systems they target.

Circom

Circom [4] is one of the most popular ZKP DSLs and the main focus of experiments in this project. It targets R1CS-based proof systems, most notably Groth16. Circuits in Circom are composed of signals (variables), components (reusable subcircuits), assertions (runtime hints) and algebraic constraints over a finite field. Circom is intentionally very low-level: all constraints must be **explicitly** written in a valid R1CS form. This means that, only constraints of the form $a \cdot b = c$ where a, b and c are linear are permitted. More complex polynomial expressions must be decomposed manually into multiple constraints.

A particularly subtle aspect of Circom is the distinction between `assert` statements and actual constraints written using ===. Assertions are checked only during witness generation and do *not* contribute to the constraint system itself. This distinction has been a recurring source of bugs and misunderstandings in real-world circuits [21], and as my analysis shows has directly affected existing compiler-testing tools (see Section 3.2.1).

During compilation, the Circom compiler produces both an R1CS constraint system and a witness generator, implemented either in JavaScript or in *C++*. The strictness and low-level nature of Circom make compiler testing challenging, as generating semantically valid Circom programs is more involved than for higher-level DSLs.

The following example illustrates how even a simple conditional computation must be encoded explicitly at the constraint level in Circom:

```
// Booleanity check component
template IsBoolean() {
    signal input in;
    in * (in - 1) === 0;
}

// Circom "Mux": out = (sel == 0) ? a : b
template Mux() {
    signal input sel;
    signal input a;
    signal input b;
    signal output out;

    component checkSel = IsBoolean();
```

```
    checkSel.in <== sel;

    out === a + sel * (b - a); // Explicitly quadratic
    // out === a + sel * b - sel * a; would not work
}

component main = Mux();
```

Circom and its compiler became the primary focus of the experimental evaluation presented in Chapter 5. While working on my testing framework, I carried out an in-depth investigation of the Circom compiler's internal architecture and implementation details. As part of this effort, I assembled a dedicated document describing the compiler architecture and compilation pipeline in considerable detail. This document is included in Appendix A.

Gnark

Gnark [6] is a Go-based framework for writing zero-knowledge circuits using native Go code. Compared to Circom, Gnark operates at a higher level of abstraction: developers can use standard Go abstractions such as helper functions, structs, and reusable APIs, while the framework automatically translates these operations into algebraic constraints. At the circuit-definition level, Gnark is largely backend-agnostic. The same circuit can be compiled to different proving systems without modifying the circuit logic. In practice, however, Gnark is most commonly used with R1CS-based proving systems. Unlike some other ZKP compilers, the Gnark compilation pipeline performs very little constraint-level optimisation. As a result, the generated constraint systems often closely reflect the structure of the original circuit implementation.

ZoKrates

ZoKrates [22] is a DSL and tooling suite designed primarily for Ethereum-oriented zkSNARK applications. It provides an end-to-end workflow, covering circuit specification, witness generation, proof generation, and the production of Solidity verifier contracts for on-chain verification [23]. ZoKrates compiles programs into an R1CS constraint system and targets multiple Groth16-style SNARKs. This DSL emphasizes integration with Ethereum smart contracts, making it a popular choice for early zkSNARK-based decentralized applications.

CirC

CirC [24] is an intermediate representation and compiler framework for zero-knowledge circuits. Conceptually, it aims to play a role similar to LLVM in traditional compilation pipelines: rather than serving as a user-facing DSL, it provides a common intermediate layer on top of which analyses, optimisations, and multiple backends can be built. Programs are translated into a lower-level circuit representation, on which CirC can perform various compiler passes and transformations before generating the final constraint system. This architecture is intended to enable reusable compiler infrastructure across different ZKP languages and proving systems. At present, CirC primarily supports ZoKrates as its frontend language and can therefore be viewed as an alternative compilation pipeline for ZoKrates programs. Instead of compiling directly through the standard ZoKrates toolchain, programs can be lowered into CirC's intermediate representation and processed through the CirC pipeline.

Noir

Noir [5] is a modern high-level DSL for writing zero-knowledge circuits. Compared to lower-level frameworks such as Circom, Noir provides significantly richer language abstractions, including structured control flow, generic functions, integer types (e.g., `u32`, `i32`), arrays, and strings [25]. The language is designed to resemble conventional programming languages more closely, while still compiling to arithmetic constraints suitable for zero-knowledge proving systems. Rather than targeting a specific prov-

ing system directly, Noir compiles programs into an intermediate representation called ACIR (Abstract Circuit Intermediate Representation). Backend implementations can then translate ACIR into concrete proving-system-specific constraints. The primary backend - Barretenberg [26] - is developed as part of the Aztec ecosystem [27], although additional backends also exist.

ACIR [28] is a backend-independent intermediate representation intended to abstract over different proving systems. Unlike traditional R1CS representations, ACIR constraints are expressed more generally as quadratic polynomial assertions rather than strictly in the canonical $a \cdot b = c$ R1CS form. In addition to arithmetic assertions, ACIR also supports *black-box functions*. These represent higher-level operations that are delegated to the proving backend rather than expanded directly into arithmetic constraints at the ACIR level. A common example is the `RANGE` black-box operation, which constrains a value to fit within a specified bit-width. Instead of encoding bit-decomposition constraints directly inside ACIR, the backend is expected to provide an efficient implementation of the corresponding range-check gadget. ACIR additionally supports opcodes for unconstrained execution. These allow portions of computation to be performed outside the constraint system, similarly to witness-generation hints in other DSLs. Such operations do not directly contribute constraints.

o1js

Mina Protocol is a blockchain designed to stay a constant in size by using zero-knowledge proofs so anyone can verify the network quickly without downloading its full history. The Mina ecosystem uses `o1js`, a high-level TypeScript/JavaScript library for writing Mina zkApps and their associated SNARK-friendly circuits [29, 30]. Developers express application logic using high-level constructs, which are then compiled into circuits targeting a PLONK-like proving system. Mina places strong emphasis on recursion and succinctness, resulting in deeply layered compilation pipelines and recursive proof composition [31].

Cairo

Cairo is StarkWare's programming language for provable computation, designed around STARKs [32, 33]. Programs execute on the Cairo virtual machine and are proven using an execution trace and AIR-style constraint system. Unlike the other languages discussed here, Cairo does not compile to an arithmetic circuit in the traditional sense. Instead, correctness is proven over the execution trace of the VM. Similarly to Circom's assertions, Cairo also supports *hints*, which influence execution but are not directly constrained.

Leo

Leo is a statically typed, imperative language for writing private applications on the Aleo blockchain [34, 35]. Leo programs compile into AleoVM instructions, which are subsequently lowered into arithmetic constraints and proven using Aleo's native proving system, Varuna [36].

2.3 Automated Testing Techniques

This section introduces the automated software testing techniques relevant to the evaluation of ZKP compilers. Particular emphasis is placed on fuzzing and oracle design, as these are necessary to understand the existing ZKP compiler testing tools and the methodology used throughout this work.

2.3.1 Fuzzing

Fuzzing is an automated software testing technique that repeatedly executes a program under test using automatically generated or mutated inputs in order to trigger unexpected behaviour such as crashes, assertion violations, incorrect outputs, or performance problems [37, 38].

Modern fuzzers can broadly be divided into mutation-based and generation-based approaches. Mutation-based fuzzers derive new test cases by modifying existing inputs, often using random or structure-aware

transformations. In contrast, generation-based fuzzers construct inputs from scratch according to a grammar, schema, or other specification. Generation-based approaches are particularly important in compiler testing, where randomly generated programs must remain syntactically valid in order to reach deeper compilation stages [39, 40].

A major development in fuzzing was the introduction of coverage-guided fuzzing, where runtime feedback is used to guide exploration toward previously unseen execution paths. Coverage-guided fuzzers such as AFL instrument the target program and retain inputs that increase code coverage [41]. Such techniques substantially improve exploration efficiency compared to purely random testing.

Compiler fuzzing introduces additional challenges compared to traditional fuzzing. Randomly generated programs must frequently satisfy not only syntactic but also semantic constraints in order to avoid immediate rejection by the compiler front-end. As a result, compiler testing tools commonly employ grammar-aware generation, semantics-preserving mutations, and structured program transformations [40, 9].

In the context of ZKP compilers, fuzzing becomes particularly challenging due to the presence of arithmetic circuits, finite-field semantics, witness generation, and backend proving systems. A generated program must not only compile successfully, but also admit satisfying witnesses and valid proofs in order to exercise deeper stages of the compilation and proving pipeline.

2.3.2 Testing Oracles

A testing oracle is a mechanism used to determine whether the behaviour of a program under test is correct for a given input [26]. In traditional software testing, an oracle may simply compare a program output against an expected value. In compiler testing, however, determining correctness is more difficult, particularly when testing randomly generated programs for which no ground-truth output is available. This challenge is commonly referred to as the *oracle problem* [26].

The simplest form of compiler testing oracle is a crash oracle, which checks whether compilation terminates successfully without triggering crashes, assertion failures, or other abnormal behaviour. While effective for detecting stability and memory safety issues, crash oracles are generally insufficient for identifying logical miscompilations.

Differential testing addresses this limitation by executing multiple implementations of the same specification on the same input and comparing their outputs [42]. Any discrepancy between the implementations may indicate a bug in at least one of them. In compiler testing, differential testing may compare multiple compilers, optimisation levels, interpreters, or proving backends.

Metamorphic testing instead verifies whether expected semantic relationships hold across transformed inputs [43]. Rather than checking the correctness of a single output directly, metamorphic testing applies semantics-preserving transformations to a program and verifies that the observable behaviour remains consistent. This is particularly useful in domains where obtaining a reference output is difficult.

Compiler testing systems additionally can employ semantic oracles that reason directly about intermediate representations or generated artefacts. Such approaches can verify properties including satisfiability preservation, semantic equivalence, or constrainedness of generated constraint systems. These techniques are worth exploring in ZKP compiler testing, where a compiler may successfully produce a circuit or proof system artefact while still silently introducing semantic errors.

2.3.3 Program Generation

The effectiveness of fuzzing depends heavily on the quality and diversity of generated test inputs [38]. Fuzzers can differ substantially in how they construct new programs or mutate existing ones.

Grammar-based generation constructs inputs according to a formal grammar or language specification. This approach is particularly useful when testing compilers and interpreters, as it allows generated

programs to remain syntactically valid while still exploring diverse program structures [39].

Semantics-aware generation extends this idea further by incorporating additional semantic constraints into the generation process. In compiler testing, generated programs may need to satisfy typing constraints, variable scoping rules, or other language-specific invariants in order to exercise deeper compilation stages. Tools such as CSmith additionally avoid undefined behaviour in order to ensure that observed discrepancies correspond to genuine compiler bugs rather than invalid source programs [40]. Structured mutation techniques modify existing valid programs while attempting to preserve important structural or semantic properties. Such approaches are commonly used in metamorphic and differential testing frameworks, where preserving semantic validity is often necessary for meaningful oracle evaluation [9].

Structured program generation is important in ZKP compiler testing due to the restricted structure of valid arithmetic constraints and the need to generate programs that admit satisfying witnesses. Naive random generation frequently produces trivially unsatisfiable circuits or circuits that lack constraints that exercise only limited portions of the compilation pipeline.

2.3.4 Test-Case Reduction

Automatically generated test cases are often large, complex, and difficult to analyse manually. When a generated input triggers a bug, it is therefore useful to reduce the input while preserving the failing behaviour. This process is commonly referred to as *test-case reduction* or *delta debugging* [44].

Test-case reduction repeatedly simplifies a failing input and checks whether the bug still occurs. The goal is to produce a minimal reproducing example that is easier to understand, debug, and report. Reduction techniques are commonly used together with fuzzers, as fuzzing often produces highly complex inputs that are difficult to inspect manually.

Reduction tools are valuable in compiler testing because generated programs may trigger failures only after complex optimisation or lowering passes. Producing a small reproducing example substantially simplifies bug analysis and reporting.

2.4 SMT Solvers

Satisfiability Modulo Theories (SMT) solvers are automated tools for checking whether logical formulas are satisfiable under a combination of boolean logic and additional theories, such as arithmetic or bit-vectors. They are widely used in program analysis, formal verification, and automated testing, where reasoning about both control flow and data values is required [45].

2.4.1 SAT Solvers

At a high level, SMT solvers build on the core ideas of boolean satisfiability (SAT) solving and extend them with theory-specific reasoning. SAT solvers determine whether a propositional logic formula can be satisfied by some assignment of boolean variables. In practice, SAT queries are expressed in *conjunctive normal form* (CNF), that is, as a conjunction of clauses:

$$\varphi = \bigwedge_{i=1}^{m} C_i, \quad C_i = \bigvee_{j=1}^{k_i} \ell_{ij}$$

where each literal ℓ_{ij} is either a boolean variable x or its negation $\neg x$.

A *model* of a SAT formula is an assignment

$$M : \{x_1, x_2, \ldots, x_n\} \to \{\text{true}, \text{false}\}$$

such that the formula evaluates to true under M, denoted by:

$$M \models \varphi$$

For example, the formula

$$(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$$

is satisfied by the assignment $M(x_1) = \text{true}, M(x_2) = \text{false}, M(x_3) = \text{true}$.

2.4.2 SMT Solvers

SMT solvers generalize SAT solving by allowing formulas to contain symbols interpreted over background theories, such as linear integer arithmetic, bit-vectors, arrays, or uninterpreted functions [45]. The most popular state-of-the-art SMT solvers are Z3 [46] and CVC5 [47]. Instead of reasoning purely about boolean variables, an SMT solver decides the satisfiability of formulas in which boolean structure is combined with theory-specific constraints.

A *theory* defines a domain of values and a set of interpreted functions and predicates. Common theories include:

- Linear integer and real arithmetic,
- Fixed-width bit-vectors,
- Arrays.

An SMT formula may contain both boolean connectives and theory atoms, such as:

$$(x - y > 2) \wedge (y = x + 7) \wedge (y < 5)$$

A *model* for an SMT formula assigns concrete values to variables from the appropriate domains (e.g., integers or bit-vectors) such that all constraints are satisfied [45].

Finite field theory is particularly relevant in the context of zero-knowledge proof systems, as ZKP compilers ultimately generate constraints over a prime field. Finite field SMT theories model variables as elements of a finite field $\mathbb{F}_p$, where arithmetic operations are performed modulo a prime p. The theory supports operations such as field addition, multiplication, and equality over field elements.

At the time of writing, native support for finite field reasoning in SMT solving is still relatively experimental. In particular, cvc5 currently provides native support for finite field SMT theories [47]. This functionality enables SMT-based reasoning directly over algebraic constraint systems similar to those used in many ZKP frameworks.

2.4.3 Testing Toolkit

There is existing research devoted to developing automated tools for systematically testing SMT solvers. These tools are used to expose crashes, soundness bugs, performance issues, and inconsistencies between solver implementations. The techniques which are used include grammar-based generation, mutation, delta debugging, and semantics-preserving transformations of SMT formulas.

YinYang

One prominent family of SMT testing tools is the YinYang framework [9, 48, 49], which focuses on automatically generating and mutating SMT formulas while preserving syntactic correctness and, in some cases, semantic properties. The framework consists of three complementary techniques:

- **OpFuzz.** [49].
  It performs type-aware operator mutation, replacing operators with others of the same type. This preserves well-typedness while exercising different solver code paths.
- **TypeFuzz.** [48]
  It extends this idea with generative, type-aware mutations, constructing new sub-expressions using SMT grammar and type information. This produces more structurally diverse and complex formulas than simple operator replacement.

- **Semantic Fusion.** [9]
  Fusion combines two formulas into one in a satisfiability-preserving way, creating more complex inputs that expose interactions and corner cases absent from individual seeds. It can be viewed as a mechanism for amplifying a small set of SAT or UNSAT benchmarks into a large number of structurally diverse test cases while preserving the desired satisfiability property.

Example (Semantic Fusion) Let ϕ_1 and ϕ_2 be two seed formulas over integers:

$$\phi_1 \;=\; (x > 0) \wedge (x < 10), \qquad \phi_2 \;=\; (y > 5) \wedge (y < 20).$$

Semantic fusion merges the seed formulas and introduces fresh variables defined by a *fusion function*, for example

$$z := x + y.$$

Some occurrences of existing variables are then rewritten using the inverse of the fusion function, for example replacing x by $(z - y)$ or y by $(z - x)$. The resulting fused formula could therefore take the form

$$\phi \;=\; (z - y > 0) \wedge (x < 10) \wedge (z - x > 5) \wedge (y < 20).$$

Satisfiability (or unsatisfiability) is preserved by construction, while the solver must now reason about a new coupling between two previously independent seed formulas.

ddSMT

ddSMT [50] is a delta-debugging and reduction tool designed for SMT formulas. Given a formula that triggers a solver bug, crash, or unexpected behaviour, ddSMT attempts to minimise the input while preserving the original failure condition. The tool repeatedly applies semantics-aware simplifications and checks whether the reduced formula still reproduces the observed behaviour. This process can remove unnecessary assertions, simplify expressions, inline variables, or eliminate entire subterms. Producing smaller reproducing examples significantly simplifies debugging and root-cause analysis.

ET

ET (Enumerative Testing) [51, 52] is a bounded exhaustive testing framework for SMT solvers based on grammar-driven enumeration. Instead of randomly mutating formulas, ET systematically enumerates formulas from a context-free grammar, typically generating smaller formulas first.

The approach is motivated by the *small-scope hypothesis*, which states that many software bugs can be triggered by relatively small inputs [53]. By systematically enumerating formulas within a bounded size, ET provides a structured exploration of the input space in contrast to purely random fuzzing approaches.

For example, a very small grammar for arithmetic formulas could be:

$$\begin{aligned} E &::= x \mid y \mid 0 \mid 1 \mid (E + E) \mid (E * E) \\ B &::= (E = E) \mid (E < E) \end{aligned}$$

From this grammar, ET could systematically generate formulas such as:

$$\begin{aligned} &(x = 0) \\ &((x + 1) = y) \\ &((x * y) < (y + 1)) \\ &(((x + y) * 1) = (x + y)) \end{aligned}$$

Rather than relying on random mutations, the enumeration process guarantees that all formulas up to a certain structural size are explored.

ET supports grammar-based generation for multiple SMT-LIB theories and can be used together with differential testing across multiple solvers or solver versions. The tool was used to study both correctness and performance evolution of SMT solvers across multiple releases [51].

SMTFuzz

Like ET, SMTFuzz [54, 55] is a grammar-based testing tool for SMT solvers. However, instead of systematically enumerating formulas, it generates random SMT-LIB programs by sampling expressions from theory-specific grammars.

The tool supports a wide range of SMT logics and theories, including integer, bit-vector, string, and array theories, and has been successfully used to discover soundness bugs and crashes in several SMT solvers.

Chapter 3

Exploration and Enhancement of Existing ZKP Testing Tools

In this chapter, I examine existing work on testing ZKP compilers and use it as the starting point for the experimental part of the project. This is still a relatively young research area and, to the best of my knowledge, there exist only two publicly available tools specifically designed for systematic testing of ZKP compiler pipelines: *MTZK* [8] and *Circuzz* [7]. Both frameworks combine fuzzing with metamorphic testing to detect runtime failures and logical inconsistencies in compiled circuits, but they differ in their program transformations, target languages, and correctness oracles.

The chapter first analyses these two frameworks, with particular emphasis on *Circuzz*, which became the main basis for my initial experiments. I then describe the extensions implemented during this exploratory phase, including improvements to the Circom generator, experiments with a Picus-based oracle, and support for additional backends. The chapter concludes by explaining how the limitations observed in these tools motivated the SMT-based framework introduced in the following chapters.

Although some of this material could be viewed as background (in particular Section 3.1.1), I include it here because it is specific to the experimental work in this chapter: the analysis of MTZK and Circuzz directly motivates the modifications to Circuzz and the design choices explored in the rest of the project.

3.1 Existing ZKP Testing Frameworks

3.1.1 MTZK [8, 56]

MTZK proposes a testing framework based on generating random programs in a custom intermediate representation (IR). The IR is designed to capture common constructs found in ZKP domain-specific languages, including arithmetic and boolean expressions, assignments, conditionals, loops, assertions, and variables annotated with public or private visibility. The IR also includes a simple type system supporting finite field values, integers, unsigned integers, and booleans. Programs are generated recursively using grammar-based rules together with type-guided expression generation to ensure that generated expressions are well-typed and semantically valid.

Importantly, the initially generated programs do not contain explicit constraints, program inputs, or outputs. Instead, MTZK first generates ordinary computations and only later introduces constraints and visibility mutations through metamorphic transformations.

Figure 3.1 illustrates the overall testing pipeline of MTZK. Programs are first generated in the IR and executed concretely using randomly generated input values. Programs that trigger runtime errors are discarded. The remaining programs are then translated into a target ZKP language. MTZK supports ZoKrates [22], Noir [5], Cairo [33] and Leo [34]. MTZK applies two kinds of mutations. The first kind

adds constraints derived from concrete executions of the generated program. The tool first executes the program and records the runtime values of intermediate variables. It then injects new assertions that are guaranteed to hold for the observed execution trace. For example, if a variable `x` evaluates to `76`, MTZK may insert constraints such as `assert(x == 76)` or logically equivalent predicates such as `assert(!(x < 76))`. It also generates tautological constraints and compositions of constraints using conjunctions and disjunctions. These inserted assertions are intended to remain satisfiable by construction while exercising the compiler's constraint generation and optimisation pipeline. The second kind of mutation replaces constants and expressions with explicit program inputs, allowing the tool to control which values are supplied during witness generation. The oracle used by MTZK is simple. All generated programs are expected to be satisfiable, so the tool checks only for compilation failures, crashes, or cases where a circuit that should be satisfiable is instead reported as unsatisfiable. This design allows MTZK to find crashes and certain completeness-related issues, particularly cases where valid programs are incorrectly rejected after compilation or optimisation.

Bugs uncovered. MTZK was evaluated on four ZKP compilers and the authors report discovering 21 distinct bugs, consisting of both compilation failures and logic errors. The paper presents an example of a crash in Cairo, where an error in variable liveness analysis causes the compiler to incorrectly mark a variable as inactive, leading to an invalid memory access during constraint generation and ultimately a compiler crash.

More importantly, the paper highlights one logic error that got propagated all the way to the prover code. This bug is in Noir and is caused by an incorrect SSA (Single Static Assignment) transformation. In this case, an underflow operation is mishandled, resulting in an incorrect value being propagated through the compiled program. Although the source program semantics imply that a constraint should be satisfiable, the compiled circuit produces a proof that is rejected by the verifier. In other words, a valid execution (with a correct witness) leads to an UNSAT outcome after compilation. As such, this bug can be interpreted as a completeness violation.

Unfortunately, beyond the two detailed case studies presented in the paper, relatively little technical detail is provided for the remaining discovered bugs. While the authors report aggregate statistics and discuss several exploit scenarios, the paper does not provide issue tracker references, patches, or minimal reproducing examples for most of the reported bugs. Such information is also difficult to locate in the accompanying code artefact. As a result, independently validating the precise root causes, severity, and practical impact of many of the reported issues remains challenging.

Limitations. A further limitation is that the metamorphically modified programs are only evaluated using the concrete witness values generated during execution. The framework does not systematically investigate the resulting constraint systems themselves or search for alternative satisfying assignments. Consequently, the testing process validates only that the expected witness is accepted, rather than checking whether the compiled circuit admits additional unintended witnesses or violates the intended semantics of the source program.

Threats to validity. The accompanying code repository is not in a usable state, as it does not compile without significant modification. As a result, it is challenging to evaluate the tool in practice or validate the effectiveness of the proposed approach. Additionally, the paper provides limited support for independently verifying its results. In particular, it does not include concrete references to the reported bugs, making it difficult to fact-check the claims or reproduce the findings.

3.1.2 Circuzz [7, 57]

Circuzz is a fuzzing framework for zero-knowledge proof compilers that supports Circom [4], Gnark [6], Noir [5], and Corset [58]. The Corset project is currently archived. Circuzz tests the end-to-end behaviour of the pipelines including compilation, witness generation, proof generation, and verification.

Instead of generating constraint-free programs, Circuzz constructs circuits in a custom intermediate

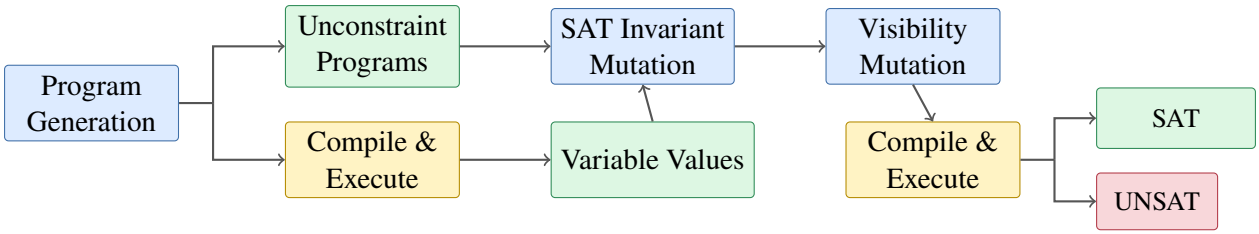

Figure 3.1: Visualisation of the MTZK pipeline.

language. Each generated program explicitly defines inputs, outputs, output assignments, and assertions. Programs are generated either over boolean values or over finite fields, depending on the target backend.

The high-level workflow of Circuzz is shown in Figure 3.2. The tool generates random circuits, applies program transformations, compiles the resulting programs, executes them on generated inputs, and compares their observable behaviour at each stage of the pipeline.

Circuzz allows generated programs to be syntactically or semantically invalid, and does not require all programs to successfully compile. Inputs are generated using a black-box strategy, with a bias towards certain "interesting" values and are not guaranteed to satisfy the circuit, which increases input diversity compared to MTZK.

The tool applies two classes of transformations. *Equivalence* transformations rewrite expressions in a way that should preserve program semantics. *Weakening* transformations relax the program so that the transformed version should accept a superset of the inputs accepted by the original. The paper does not mention the weakening transformations; however, they are visible in their code repository [57].

Circuzz employs two oracles. A metamorphic oracle checks that the original and transformed programs behave consistently at each stage: both should either compile or fail, both should either generate a witness or not, and both should either successfully verify or reject a proof. A second oracle enforces basic consistency between stages, for example requiring that proof generation succeeds whenever witness generation succeeds.

Bugs uncovered. Circuzz was evaluated on four ZKP pipelines and discovered a total of 16 bugs, of which 15 were confirmed and fixed by developers. Unlike MTZK, the authors provide detailed descriptions, minimised examples, and references for each reported bug, making their results transparent and reproducible. Seven of the reported bugs occur during the witness generation stage. These bugs manifest either as failures during witness generation or as incorrect witnesses that do not properly satisfy the underlying constraint system. In both cases, the resulting witness cannot be used to successfully generate or verify a proof, effectively breaking the correctness of the pipeline. Four of the reported bugs occur in the Corset check stage, which is an optional validation step that evaluates whether a given assignment satisfies the generated constraint system; however, since the Corset project is no longer actively maintained, the relevance of these bugs is limited. All compilation-stage bugs manifest as compile-time failures, such as crashes or rejected programs, and do not result in the generation of incorrect or underconstrained constraint systems. Concretely, these are: a compiler panic due to an unchecked casted branch, a stack overflow caused by deeply nested less-than expressions, and inconsistent handling of binary decomposition requiring at least one bit. Two of the discovered bugs occur in the prover stage, and both correspond to implementation issues in the prover that cause the proof-generation algorithm to fail or crash rather than producing incorrect proofs.

Limitations. While Circuzz can expose behavioural differences between programs, it does not determine which version exhibits the correct behaviour. If one program is satisfiable and the other is not, it is unclear whether this corresponds to a soundness bug or a completeness bug without manual inspection. Furthermore, Circuzz focuses on observable compilation and execution outcomes and does not analyse the generated constraint systems directly.

Another limitation is that Circuzz evaluates programs on randomly generated private inputs. This

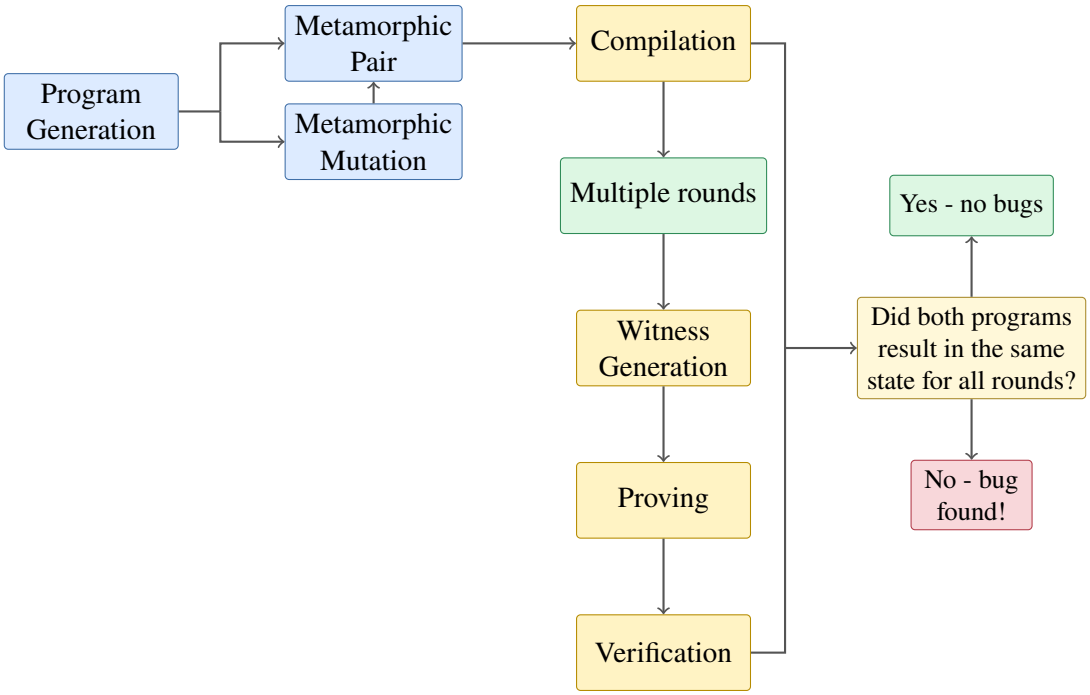

Figure 3.2: Simplified visualisation of the Circuzz pipeline.

introduces a high degree of randomness, while the probability of sampling semantically interesting inputs - such as those that should satisfy the constraints but do not, or vice versa - is low. As a result, the approach may miss subtle bugs that only manifest on carefully structured inputs, effectively limiting its ability to detect deeper soundness or completeness issues due to over-reliance on random testing.

An additional limitation arises in the Circom backend. Generated programs frequently rely on `assert` statements rather than constraints written using ===. Since assertions are checked only during witness generation and do not contribute to the constraint system a big part of the compiler remains unexplored by their generated inputs. This observation directly motivates the initial direction of my work (see Section 3.2.1.

3.1.3 Framework Comparison

Both MTZK and Circuzz aim to detect bugs in ZKP compilers using fuzzing combined with metamorphic testing, but they differ in how programs are generated and how correctness is evaluated.

MTZK generates constraint-free programs and then injects constraints derived from concrete executions. Its oracle assumes that all generated programs should be satisfiable and checks whether the compiled circuit behaves accordingly. In contrast, Circuzz generates full circuits upfront, including inputs, outputs, and constraints, and compares the behaviour of original and transformed programs across the entire pipeline (compilation, witness generation, proving, and verification).

Despite these differences, both tools share an important limitation that comes directly from the black-box nature of their oracles: neither of them directly inspects the generated constraint systems. Instead, they rely on executing programs on specific inputs and observing the resulting behaviour. As a consequence, both approaches may miss bugs that only manifest under particular inputs or that affect the structure of the constraint system without immediately impacting execution outcomes.

The design of MTZK further restricts the class of bugs it can detect. Since all generated constraints are constructed to be satisfiable and the oracle only checks for unexpected UNSAT results, MTZK effectively targets completeness violations. It cannot detect cases where a compiler produces an underconstrained circuit that accepts invalid witnesses, i.e., soundness violations.

Circuzz, on the other hand, in principle allows for detecting both soundness and completeness issues,

since it compares behaviour between semantically related programs and does not assume satisfiability. However, in practice, its reliance on randomly generated inputs makes the probability of triggering such bugs relatively low, especially for subtle constraint-level issues that require carefully structured inputs.

Overall, both tools are proven to be effective at detecting crashes, but their black-box nature and reliance on concrete execution limit their ability to reason about the correctness of the underlying constraint systems.

3.2 Enhancing Circuzz

For the initial experimental part of this project, Circuzz was chosen as the primary tool for in-depth analysis and experimentation. The main reason for this choice was practical: the MTZK artefact was not in a usable state. The focus of this project later shifted towards the design of a custom testing approach. As a result, MTZK was not explored in depth beyond its conceptual design and reported results.

3.2.1 Improving the Circom Generator

While experimenting with the tool one limitation immediately stood out to me. The original Circuzz generator for Circom wrapped assertions inside `assert` blocks. However, `assert` in Circom is only a hint to the witness generator and does not translate into constraints in the compiled R1CS. As a result, the generated programs did not effectively exercise the *constraint generation* component of the compiler. In practice, many generated programs contained very few actual constraints. The only constraints arose implicitly from assignments to intermediate signals. For example:

```
template Example() {
    signal input x;
    signal y;
    signal z;

    y <== 5;
    z <== y;

    assert(x == z);
}
```

Here, the `assert` statement does not contribute to the R1CS. The only constraints enforce that `y` is a constant and `z = y`, while `x` remains unconstrained. The resulting constraint system is therefore extremely small and underconstrained. This limits the effectiveness of compiler testing, as it fails to exercise large parts of the pipeline responsible for generating and optimising constraints. In particular, optimisations on non-trivial algebraic expressions don't get many opportunities to get triggered.

This limitation is reflected in the results for the Circom backend. Out of the four bugs reported for Circom, three were found in the witness-generation stage - the stage to which the code inside the 'assert' statements contributes. The last bug was found in the prover stage [7].

Furthermore, this behaviour does not appear to be intentional. In the Circuzz intermediate language (CircIL), assertions are explicitly treated as constraints; for example, in the paper the authors state that `assert(in0 != in1)` "introduces the constraint that `in0 != in1`" [7] which is not true. This semantic mismatch means that the intended semantics of CircIL assertions are not preserved in the generated Circom programs. As a result, the generated circuits often fail to exercise the parts of the compiler responsible for constructing and optimising non-trivial constraint systems.

Solution

A natural first attempt to address this limitation is to simply unwrap all assertions and translate them directly into constraints. However, this is not sufficient in the case of Circom as it requires constraints

to be expressed explicitly in R1CS form. As a result, arbitrary expressions appearing inside assertions cannot always be translated directly into valid constraints. In particular, higher-degree polynomials must be decomposed into a sequence of quadratic constraints using auxiliary variables. To address this, I extended the Circom generator to recursively decompose non-quadratic expressions into valid R1CS constraints. The key idea is to introduce intermediate signals that capture partial computations, ensuring that each generated constraint fits within the R1CS model.

For example, consider the expression

$$x^3 + 1.$$

This cannot be represented directly as a single R1CS constraint. Instead, it can be rewritten using auxiliary variables:

$$t_1 = x \cdot x,$$
$$t_2 = t_1 \cdot x,$$
$$t_2 + 1 = 0.$$

Each step corresponds to a valid quadratic constraint, and together they faithfully encode the original expression.

An alternative approach would be to restrict the generator to produce only quadratic expressions from the outset. While this would simplify constraint generation, it would significantly limit the applicability of metamorphic testing. The reason is that some important metamorphic transformations do not preserve quadratic form. A key example is distribution:

$$((a + b) \cdot c) \longrightarrow (a \cdot c) + (b \cdot c).$$

Although both expressions are semantically equivalent, the left-hand side can be represented as a single quadratic constraint, whereas the right-hand side contains multiple multiplicative terms and therefore requires decomposition into several R1CS constraints. Restricting generation to quadratic expressions would therefore eliminate a number of useful metamorphic relations and reduce the diversity of generated program pairs. Nevertheless, for completeness, I also evaluated a generator restricted to quadratic expressions only, however, this approach did not yield useful results.

Restricting Inequality-Based Metamorphic Relations

During the development of the generator, it became necessary to remove a subset of metamorphic relations that transform equalities into inequalities, such as:

$$a = b \longrightarrow (a \leq b) \wedge (a \geq b).$$

While such transformations are logically valid over the integers, they are problematic in Circom due to the way inequalities are implemented in `circomlib` [59]. Circom does not provide a native notion of ordering over finite fields. Instead, comparisons such as $a < b$ or $a \leq b$ are implemented using bit-decomposition circuits. The standard `circomlib` implementation (e.g. `LessThan`) converts field elements into bounded binary representations and compares them bitwise. The following code snippet reproduces the `LessThan` template implementation from `circomlib` [59]:

```
template LessThan(n) {
    assert(n <= 252);
    signal input in[2];
    signal output out;

    component n2b = Num2Bits(n+1);

    n2b.in <== in[0]+ (1<<n) - in[1];

    out <== 1-n2b.out[n];
}
```

These constructions rely on the assumption that inputs fit within a fixed bit-width. In the BN254 field used by Circom, `circomlib` restricts comparisons to at most 252 bits - two bits smaller than the field modulus - to avoid overflow issues and ambiguities in the bit decomposition. Consequently, equivalences such as

$$a = b \iff (a \leq b) \wedge (a \geq b)$$

are only guaranteed within this restricted range. Including such metamorphic relations would therefore introduce counterexamples caused by the encoding of inequalities rather than by compiler bugs. For this reason, all metamorphic relations involving inequalities were removed from the generator.

Improving Boolean Constraint Generation

As mentioned previously, Circuzz also contains a Boolean-based generator. It suffered from the same issue as arithmetic constraint generation: Boolean conditions were emitted inside `assert` statements and therefore did not contribute constraints to the compiled R1CS.

To address this issue, I modified the generator to translate Boolean assertions into dedicated Boolean helper circuits from `circomlib`. In Circom, Boolean values are represented as field elements constrained to the set $\{0, 1\}$. Logical operations can therefore be expressed using arithmetic constraints over finite field elements.

These circuits implement Boolean operations over field elements by constraining values to behave as bits. Conceptually, this amounts to using arithmetic encodings such as $c = a \cdot b$ for AND, $c = a + b - a \cdot b$ for OR, and $c = 1 - a$ for NOT, together with the Booleanity constraint $b \cdot (b - 1) = 0$.

Results

To evaluate the impact of the modified generator, I conducted a 24-hour fuzzing campaign using the improved Circom backend - both the arithmetic and boolean versions.

During this campaign, the generator was able to uncover two additional bugs in the Circom pipeline. These bugs were not detected by the original Circuzz configuration. A detailed description of these bugs and the campaign, is provided in Section 5.3.

3.2.2 Integrating Picus into the Testing Oracle

As explained previously one of the main limitations and possible areas of improvement I saw in *Circuzz* was that its oracle does not inspect the generated constraints. To improve the testing framework in that direction, I initially tried extending the oracle with an additional check based on Picus. In the extended oracle, constrainedness is treated as an invariant: for a pair of semantically equivalent programs (C_1, C_2), both should be either well-constrained or underconstrained. A mismatch is reported as a potential bug.

Implementation and Observations

The Picus check was implemented for both Circom and Gnark, as these are directly supported by Picus. For each generated circuit, the constraint system was analysed after compilation, and the result was incorporated into the oracle.

However, experiments quickly revealed an important limitation. Since the Circuzz generator produces highly random circuits with very little structure, many generated programs are either trivially underconstrained or Picus times out on them (classified as inconclusive). Large parts of the input space often remain unrestricted, and the resulting constraint systems exhibit obvious non-determinism.

This behaviour is illustrated in Table 3.1, which shows the distribution of constrainedness classifications across generated programs.

The additional Picus analysis also introduces a substantial runtime overhead. Furthermore, inconclusive results are particularly expensive, as they often correspond to solver timeouts after substantial analysis

Backend	Fully Constrained	Underconstrained	Inconclusive	Other
Circom	20.0%	39.7%	30.1%	10.2%
Gnark	18.5%	37.0%	29.6%	14.8%

Table 3.1: Constrainedness classification results produced by Picus during a 30 min campaign.

effort has already been spent. This further reduces the number of programs that can be tested within a fixed time budget, which was already low.

For the remaining programs, the oracle mainly searches for cases where a metamorphic transformation changes an underconstrained program into a fully constrained one. Intuitively, such cases appear relatively rare and we would rather explore the other direction. Overall, the additional oracle check combined with a highly random Circuzz generator provides limited practical benefit despite its high computational cost.

Weakly Constrained Satisfying Inputs

Another limitation of Circuzz stems from its reliance on purely random program generation. After constructing a metamorphic pair, each circuit is executed on randomly generated inputs. While the original evaluation reports that roughly 55-65% of generated inputs satisfy the circuits [7], many of these satisfying executions are structurally trivial.

The paper; however, does not analyse the quality of the circuits for which the inputs are satisfying. Due to the highly random nature of the generator, assertions are often entirely disconnected from the input variables. As a result, a satisfying assignment may not reflect any meaningful relationship involving the inputs.

To study this effect, I analysed whether assertions depended on at least one input signal. Assertions satisfying this property were classified as *connected*; all others were considered *disconnected*.

Table 3.2 shows the distribution of satisfying executions.

Backend	Connected SAT	Disconnected SAT
Circom	54.5%	45.5%
Gnark	56.9%	43.1%

Table 3.2: Structure of satisfying assertions in SAT circuits of a 30 min campaign.

These results indicate that a substantial fraction of satisfying executions do not meaningfully constrain the inputs. Together with the large number of underconstrained circuits discussed previously, this highlights a broader issue: the heavy reliance on randomness in Circuzz often produces circuits with little semantic structure.

3.2.3 Additional Backends

To further explore the generality of the Circuzz approach and gain more experience working with fuzzers, I extended the framework with two additional backends targeting ZoKrates and `o1js` (Mina). These backends allowed testing different ZKP ecosystems with distinct compilation pipelines and execution models.

ZoKrates Backend

The backend targets two curves supported by ZoKrates and uses a native `assert()` construct that directly translates into constraints.

From an architectural perspective, the ZoKrates backend integrates smoothly with Circuzz, as its execution model is the same as for previous DSLs.

The backend proved to be effective in practice. During experiments, two bugs were discovered in the ZoKrates pipeline (see Section 5.3). Notably, none of these bugs corresponded to logical inconsistencies

in constraint generation; instead, they were related to code-analysis part.

o1js (Mina) Backend

The `o1js` backend differs slightly from the other backends due to the way Mina programs are represented and executed.

Unlike previously explored systems, `o1js` does not produce standalone intermediate artefacts that can be stored and reused between stages. Instead, circuits are defined directly as TypeScript programs and remain inside the running Node.js process. As a result, the backend uses a single long-lived execution process in which compilation, proving, and verification are performed within the same runtime environment.

In practice, however, the backend exhibited severe performance limitations. Even with batching, the compilation step remained extremely expensive, and the overall throughput was significantly lower than for other backends. Table 3.3 illustrates the observed performance.

Configuration	Avg. time/test	Tests/sec	Rel. throughput
Circom (baseline)	11.84s	0.0845	1.000×
`o1js` (batched)	197.03s	0.0051	0.060×

Table 3.3: Performance overhead of the `o1js` backend during a 30 min campaign.

The already high computational cost of ZKP pipelines is further exacerbated in `o1js`, making it difficult to achieve meaningful testing throughput. For this reason, I decided not to pursue further development of the `o1js` backend within this project and focused on other directions. While the platform is conceptually interesting, its current performance characteristics make it difficult to perform fuzzing-based testing.

3.3 Reflection

Experimenting with CIRCUZZ provided several important insights that ultimately influenced the direction of this project. Two main limitations became apparent during the experiments.

First, the generator used by CIRCUZZ produces highly random and often poorly structured programs. Many generated circuits contain weakly connected or completely disconnected assertions, large unconstrained regions, and randomly sampled inputs that do not meaningfully interact with the constraints. As a result, even satisfying executions are frequently structurally trivial and do not significantly exercise the constraint generation or optimisation stages of the compiler. Intuitively, such programs are unlikely to stress the more complex parts of the pipeline, which might be the reason why there is no logical bugs found in these stages.

Second, the oracle used by CIRCUZZ is black-box in nature. It reasons mostly about externally observable behaviour, such as whether compilation, witness generation, proof generation, or verification succeed. While this approach is effective at detecting crashes and pipeline inconsistencies, it may fail or be suboptimal at detecting logical errors inside the generated constraint system itself.

These observations motivated the remainder of this project. The central theme of the later chapters is therefore the development of techniques that reason more directly about generated constraints and their satisfiability properties.

Chapter 4

SMT-Based Testing Framework

The previous chapter showed two main limitations of the existing approaches. First, the generated programs are highly random and lack structure. Second, the testing oracle does not reason directly about the correctness of the generated constraint systems.

In this chapter I introduce an SMT-based testing tool that is designed to address both of those limitations. The idea is to generate more structured programs based on SMT formulas, and to use the solvers as an oracle that analyses the resulting constraints directly. This aims for allowing the testing process to better detect logical bugs in constraint generation.

Very late into the project, after the design, implementation, and evaluation of the main framework presented in this chapter had already been completed, I revisited the Noir language from a different perspective. This led to a extension and divergence from the original implementation and a broader investigation of ACIR-based testing. The resulting design and implementation are presented at the end of this chapter.

The chapter is organised as follows. Section 4.1 first presents the overall architecture of the framework and semantic properties checked by the oracle. Section 4.2 then describes how SMT benchmarks are generated, including the use of YinYang, generating properly constrained programs, and the different SMT theories explored. Section 4.3 explains how generated formulas are translated into ZK DSLs and compiled into constraint systems. The optimisations used to make the oracle faster are discussed in Section 4.4, followed by several further implementation topics in Section 4.5. Finally, Section 4.6 describes the previously-mentioned late-stage extension of these ideas to Noir and ACIR.

4.1 Architecture Overview

The core idea of the tool is to verify whether the constraint systems produced by a ZKP compiler preserve important semantic properties of the original SMT formulas.

The tool operates in three distinct modes:

- **Satisfiability preservation.**
  This mode checks whether formulas that are satisfiable before compilation remain satisfiable after translation into the target constraint system.
- **Unsatisfiability preservation.**
  Similar as the previous one but checks whether unsatisfiable formulas remain unsatisfiable after compilation, helping detect missing or incorrectly generated constraints.
- **Constrainedness preservation.**
  It uses a Picus-based oracle to analyse whether the constrainedness properties of the generated circuit are preserved across compilation.

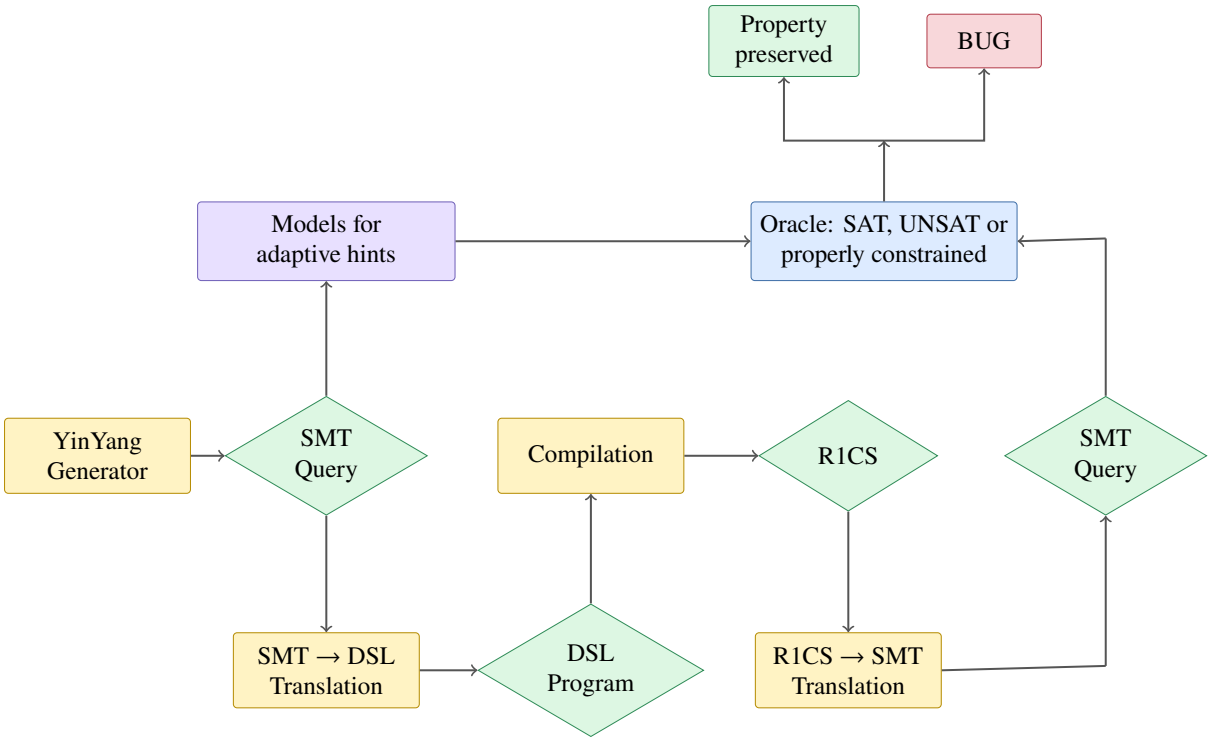

Figure 4.1: High-level architecture of the SMT-based testing approach.

Together, these modes aim to detect semantic inconsistencies introduced during constraint generation and compilation. The overall architecture of the tool is shown in Figure 4.1.

The pipeline consists of three main components:

- **Benchmarks and Program Generation.**
  The job of the generator is to provide us with SMT formulas used for testing. The main challenge is to generate programs with controlled properties. In particular, we want to produce both satisfiable and unsatisfiable programs, and in the case of the Picus-based oracle, programs that are meaningfully constrained.
- **Translation and Compilation.**
  The generated programs are translated into the target DSL (e.g. Circom or Gnark) and compiled into constraint systems. The main challenge in this step is to ensure that the semantics of the SMT formulas are preserved during translation.
- **Oracle.**
  The oracle analyses the resulting constraint systems. In the SMT-based oracle, the goal is to check whether satisfiability is preserved, i.e. whether the compiled constraints are satisfiable or unsatisfiable as expected. In the Picus-based oracle, the goal is to check whether constrainedness is preserved, i.e. whether the circuit is well-constrained or underconstrained.

The remainder of this chapter provides an overview of each of these components, together with the main implementation challenges and design decisions involved in building the framework.

4.2 Benchmarks and Program Generation

4.2.1 Fusion-based Generator

To generate structured programs, I use SMT formulas produced by semantic fusion from YinYang (described in Section 2.4.3). The choice of YinYang is motivated by the following observations:

- **Controlling semantic properties through fusion.**
  Fusion provides a mechanism for controlling important semantic properties of the generated programs. By introducing or removing dependencies between variables, the generator can influence whether a formula is satisfiable or unsatisfiable. This addresses one of the main challenges of the framework: generating programs with controlled semantic properties that can later be checked by the testing oracle.
- **Seed amplification and structural preservation.**
  Semantic fusion allows a relatively small set of initial seeds to be amplified into a large number of test cases, while still preserving meaningful logical structure.
- **Similarity to SMT solving workflows.**
  The proposed framework operates in a way that is conceptually similar to an SMT solver: it takes a logical specification, translates it into a constraint system, and reasons about its properties. From this perspective, SMT-based testing techniques are a natural fit, and tools designed to find bugs in SMT solvers are likely to be effective in this setting as well.

The generated SMT formulas are translated into ZK programs. Variables in the formula are mapped to signals, and operations are translated into arithmetic constraints in the target DSL.

A design choice worth noting is the distinction between *fused* and *unfused* variables. Fusion introduces dependencies between variables by forcing them to share values. In our setting, fused variables are treated as *output signals*, while unfused variables are treated as *input signals*. Whenever possible, fused variables are introduced as witness hints rather than explicitly constrained values. This increases the variety of generated programs and may additionally exercise interactions between witness generation and the generated constraint system.

Overall, this split was introduced to encourage the use of a wider variety of language constructs in the generated programs. It also proved useful when designing the fully-constrained program generator discussed in Section 4.2.1.

Example (Input/output split in different DSLs) Consider the following SMT formula:

$$(x + y = z) \ \wedge \ (z = x).$$

Assume, the variable z is *fused*, is introduced by the fusion mechanism and x and y are normal variables.

Following the mapping described above, x and y are treated as inputs, while z is treated as an output.

A corresponding Circom program is:

```
template Example() {
    signal input x;
    signal input y;
    signal output z;

    z <- x + y; // Note that <- is used instead of <==
    z === x;
}
```

An equivalent Noir program is:

```
fn main(x: Field, y: Field) -> Field {
    let z = unsafe {
        x + y
    }; // Unsafe code does not produce constraints so it works as a hint

    assert(x + y == x);
    return z;
}
```

Picus Fusion

The Picus-based oracle requires SMT programs that are properly constrained. As a reminder, this means that, for a fixed assignment to the input variables, there should be at most one satisfying assignment to the output variables. As discussed before, fused variables are treated as outputs, while remaining variables are treated as inputs.

To generate well-constrained programs, I extend the standard YinYang approach with a modified variant of semantic fusion, which I refer to as *Picus fusion*. In the original semantic fusion technique, occurrences of both fused variables may be partially replaced using the inverse of the fusion function (see Section 2.4.3). In contrast, *Picus fusion* applies a more restrictive transformation:

- All occurrences of one fused variable are replaced using the inverse of the fusion function.
- Occurrences of the second fused variable are left unchanged.

Combined with **uniquely satisfiable** benchmark formulas, this approach produces formulas that are fully constrained according to the constrainedness definition introduced earlier.

A proof sketch of the construction is provided in Appendix B.

Example. Consider two Boolean formulas:

$$\varphi_1(a, b) = (a \leftrightarrow b) \wedge a,$$
$$\varphi_2(c, d) = (c \oplus d) \wedge c.$$

Each formula has a unique satisfying assignment:

$$(a, b) = (\texttt{true}, \texttt{true}), \qquad (c, d) = (\texttt{true}, \texttt{false}).$$

We now fuse variable a from φ_1 with variable c from φ_2. Introduce a fresh variable z and rewrite:

$$a := z \oplus c.$$

Applying this substitution to φ_1 yields:

$$((z \oplus c) \leftrightarrow b) \wedge (z \oplus c).$$

The fused formula is therefore:

$$((z \oplus c) \leftrightarrow b) \wedge (z \oplus c) \wedge (c \oplus d) \wedge c.$$

A corresponding SMT encoding is:

```
(set-logic QF_BOOL)

(declare-fun b () Bool)
(declare-fun c () Bool)
(declare-fun d () Bool)
(declare-fun z () Bool)

; rewritten 1
(assert (= (xor z c) b))
(assert (xor z c))

; unchanged 2
(assert (xor c d))
(assert c)

(check-sat)
(get-model)
```

4.2.2 Benchmark Seeds and Supported Theories

The YinYang generator operates by mutating and fusing existing SMT formulas. As a result, an initial collection of benchmark formulas is required to act as generation seeds.

Different testing oracles require different types of seed benchmarks. Standard oracles only require satisfiable or unsatisfiable formulas, while the Picus-based oracle additionally requires formulas with unique satisfying assignments.

The experiments explored three SMT theories:

- Boolean formulas
- Finite field formulas
- Integer arithmetic formulas

Boolean Theory

Boolean SMT formulas were created from existing SAT formula benchmark suites [60]. General satisfiable and unsatisfiable formulas were used for the standard metamorphic oracle.

The Picus-based oracle additionally required uniquely satisfiable formulas. These were obtained using two approaches:

- Sudoku SMT encodings from an online benchmark repository [61], as valid Sudoku instances have exactly one solution.
- Iterative model elimination: starting from a satisfiable formula, additional constraints were added to exclude previously discovered models until only one satisfying assignment remained.

Fusion for Boolean formulas was implemented using XOR, which is bijective and therefore compatible with Picus fusion. Large formulas quickly became expensive after translation into ZK constraints. To reduce benchmark size, formulas were partially evaluated using satisfying assignments: some variables were fixed to concrete model values, while the remaining variables were kept symbolic.

Finite-field Theory

After the initial Boolean experiments, I moved to finite field formulas, as finite field arithmetic directly matches the constraint systems used by most ZK frameworks.

Support for finite field SMT solving is relatively recent. The theory was introduced by Ozdemir et al. [62] and later implemented in cvc5. Unlike Boolean SMT solving, however, there are currently no publicly available FF benchmarks apart from the small set from the original paper.

As a result, I generated my own finite field benchmarks using three approaches to increase the diversity:

- **Boolean-to-field translation.**
  Existing Boolean benchmarks were translated into finite field constraints by compiling them through ZK compilers into R1CS representations, similarly to the approach used in the original finite field SMT paper [62]. While effective, the resulting formulas remained structurally close to Boolean circuits.
- **Grammar-based generation with ET.**
  ET [51, 52] was used to generate satisfiable and unsatisfiable QF_FF formulas. This produced a large number of syntactically diverse benchmarks, although many lacked meaningful algebraic structure. I further filtered them so the assertions are connected to the variables. For the generation I defined my own grammar which is included in Appendix C.
- **Poseidon-based benchmarks.**
  A custom benchmark generator based on the Poseidon permutation [63] was implemented. Poseidon is an algebraic cryptographic permutation designed for finite-field-based proof systems. It is built

from repeated rounds consisting of:

$$\text{AddRoundConstant} \;\rightarrow\; \text{S-box} \;\rightarrow\; \text{Matrix multiplication.}$$

In the first stage, a round-dependent constant is added to each state element. The S-box stage then applies a non-linear power map $x \mapsto x^{\alpha}$ to selected state elements. Finally, matrix multiplication mixes all state elements together. Unlike a hash function, which combines the Poseidon permutation with a sponge construction, the permutation itself is bijective and therefore invertible. The generated benchmarks were constructed as pre-image problems: given a fixed Poseidon output state, the SMT solver must recover the corresponding input state. The permutation was then evaluated concretely to produce an output state, which was embedded into the SMT formula as a set of equality constraints. Since the permutation is bijective, each generated benchmark has exactly one satisfying assignment, making it particularly suitable for the Picus-based oracle. The S-box exponent was fixed to

$$\alpha = 5,$$

as this is the smallest prime that preserves bijectivity over BN254. Higher values were tested but resulted in significantly worse solver performance.

Integer Theories

I also explored SMT benchmarks based on integer arithmetic (e.g. QF_LIA). This theory is attractive due to the large number of publicly available benchmarks.

However, integrating integer formulas into ZK compilers proved difficult. Although systems such as Circom, Gnark and Zokrates ultimately operate over finite fields, the main issue was not integer representation itself, but comparison operators such as:

$$<, \quad \leq, \quad >, \quad \geq .$$

These operations are typically implemented using bit-decomposition circuits. In Circom, for example, comparators from `circomlib` expand into large constraint systems.

As a result, even relatively small integer formulas produced very large R1CS instances after translation, making SMT-based reasoning prohibitively expensive. This was particularly problematic for formulas with many nested comparisons, which are common in SMT-LIA benchmarks.

Although integer theories were ultimately not used in the main evaluation, they remained an important motivation for later work on Noir support. Unlike Circom, Gnark, and ZoKrates, Noir represents many comparison operations as ACIR blackbox functions rather than immediately expanding them into bit-decomposition constraints.

Near the end of the project, I investigated Noir and ACIR in greater depth and the resulting design and implementation are discussed in Section 4.6.

4.3 Translation, Compilation & Supported DSLs

The framework is primarily designed around Circom, which serves as the main target DSL throughout most of the project. Support for additional DSLs, namely Gnark and ZoKrates (both with and without the CirC intermediate step), was added mainly to further evaluate the tool and because extending the framework to these systems was straightforward, as they ultimately compile into R1CS-style constraint systems.

The DSL-specific parts of the framework are kept relatively small. For each supported language, the implementation mainly consists of:

- translating SMT formulas into DSL programs,

- translating the resulting constraint systems back into SMT.

For program generation, the framework reuses a significant portion of the existing Circuzz infrastructure, particularly its intermediate representation (IR) and IR-to-DSL translation logic. The Circom backend incorporates the custom constraint-generation improvements described in the Section 3.2.1.

The ZoKrates backend was added as part of this work, while the Gnark backend mostly reuses the original Circuzz generation pipeline with only minor modifications.

On the constraint side, supporting multiple DSLs requires little further work, since all three systems ultimately compile into R1CS-style constraint systems. This allows the same SMT encoding pipeline to be reused across all backends with only small format-specific adjustments. Similarly, the Picus-based oracle can also support all three systems. Although Picus does not natively support ZoKrates, its constraints can be translated into an equivalent format without major changes.

4.3.1 Noir

Basic support for Noir was also added during this project. At the time of the main evaluation, the framework supported satisfiability and unsatisfiability testing for Noir programs whose ACIR representation consisted solely of arithmetic assertions. Since these assertions correspond to quadratic polynomial relations (general quadratic polynomials not just R1CS), they can be translated directly into the SMT encoding used by the other supported DSLs.

As mentioned in the introduction for this chapter, following the completion of the evaluation presented in Chapter 5, I undertook a more comprehensive study of the Noir language and its ACIR representation. This work resulted in an extension of the framework beyond the assertion-only ACIR subset used throughout the evaluation. The resulting design and implementation are presented in detail in Section 4.6.

4.4 Oracle Performance Optimisations

The previous sections described the semantic properties checked by the oracle and the translation pipeline used to obtain SMT representations of compiled constraint systems. One practical challenge of the SMT-based oracle is performance. Even small benchmarks can produce large constraint systems after compilation, particularly in DSLs such as Gnark. To improve solving time, I implemented an optimisation which I call *adaptive hint injection*. The idea is to reuse satisfying assignments obtained from the original SMT formula and inject them as partial wire assignments into the compiled R1CS before solving. Intuitively, this provides the solver with a partial solution and reduces the amount of search required during oracle execution. Hints are injected probabilistically: each hinted wire is included independently with probability p_{hint}. The framework then adapts this probability dynamically based on recent solving behaviour. If many runs time out, the amount of hinting is increased. Conversely, if solving becomes stable, the framework gradually reduces hint injection and may increase the number of extracted models used during solving. This optimisation significantly improved performance in practice, especially for large finite-field benchmarks. However, it also introduces a tradeoff: injecting hints partially fixes internal wire assignments and may reduce the solver's ability to explore alternative satisfying assignments. Intuitively, this seems acceptable, since most logical bugs are expected to arise from incorrect intermediate computations and generated constraints and should surround the correct values. The need for hinting decreased substantially on smaller benchmarks. For many DSLs, the adaptive mechanism gradually reduced the hint probability close to zero over time. Gnark behaved differently: even relatively small formulas often produced substantially larger constraint systems and therefore continued to benefit from stronger hint injection.

4.5 Further Topics

This section discusses several additional topics related to the framework development.

4.5.1 Inflating Constraint Clusters

After analysing compiler coverage during evaluation (Section D), I observed that one of Circom's optimisation passes was rarely triggered. Investigation of the compiler internals (see Appendix A) showed that this was caused by the largest linear constraint clusters remaining below the threshold required for the optimisation to activate.

A *linear cluster* is a connected component of constraints containing only linear relations between variables. Two linear constraints belong to the same cluster if they share variables either directly or transitively through other linear constraints.

For example, consider the following constraints:

$$x + y = z,$$

$$z + a = b,$$

$$b - c = 0,$$

$$c \cdot d = e,$$

$$e + f = 0.$$

The first three constraints form a linear cluster of size 3, since they are linear and connected through shared variables. Although the quadratic constraint

$$c \cdot d = e$$

is connected to the cluster through variable c, it does not contribute to the cluster size because it is not linear.

Similarly,

$$e + f = 0$$

is linear, but forms a separate cluster of size 1, since its only connection to the previous constraints passes through the quadratic relation.

To address this issue, I implemented a synthetic *constraint cluster inflation* mechanism based on removable negation chains. Before compilation, the framework injects a random number of synthetic chains into the SMT formula. Each chain introduces fresh temporary variables connected through alternating negation relations. For Boolean formulas this uses repeated `not` operations, while finite-field formulas use arithmetic negation expressions of the form:

$$1 - x.$$

The default chain length is 400 constraints, which exceeds the threshold required to trigger Circom's large-cluster optimisation pass. Importantly, including such chains does not change the satisfiability or constrainedness properties of the original formula. After compilation, all synthetic variables and associated constraints are removed from the generated R1CS to not inflate the SMT query. This is possible because the injected chains are fully determined by existing circuit variables and therefore do not add new semantic information to the system.

In short, this mechanism allows the framework to exercise optimisation paths that would otherwise remain difficult to reach using generated benchmarks alone.

4.5.2 Triggering Multiple Optimization Rounds

Another limitation identified during the coverage analysis in Section D is that most generated benchmarks appear to be fully simplified after a single optimization round. In the Circom simplification pipeline, additional rounds are triggered when previous substitutions transform existing quadratic constraints into new linear constraints, which can then themselves be further simplified.

For example, consider the constraints

$$x \cdot y = z$$

and

$$y = 1.$$

After substituting $y = 1$, the first constraint becomes

$$x = z,$$

which is linear and may itself trigger further substitutions and simplifications in subsequent rounds. In practice, most generated constraints remain either directly linear or permanently quadratic, so the optimization pipeline converges quickly. I did not identify a meaningful way of systematically increasing the number of optimization rounds while still generating semantically interesting circuits.

One possible idea was to generate long chains of Boolean-style constraints such as:

$$a_1 \vee a_2, \quad a_2 \vee a_3, \quad a_3 \vee a_4.$$

Using the standard arithmetic encoding of Boolean OR,

$$a \vee b = a + b - ab,$$

the assertion $a_1 \vee a_2 = 1$ becomes:

$$a_1 + a_2 - a_1 a_2 = 1.$$

If a previous simplification round discovers $a_1 = 0$, then substituting this into the constraint gives:

$$0 + a_2 - 0 \cdot a_2 = 1,$$

which simplifies to:

$$a_2 = 1.$$

This newly linear constraint can then be used in the next round to simplify the following constraint $a_2 \vee a_3 = 1$. However, each additional round repeats essentially the same pattern: substituting a known Boolean value into another OR expression, which then collapses into a linear constraint on the next variable. Therefore, although such chains can force multiple optimisation rounds, they do not appear to exercise meaningfully different optimisation behaviour beyond the first round. To sum up, designing benchmark-generation strategies that trigger more diverse multi-round optimisation behaviour remains an interesting direction for future work.

4.5.3 Warmup Mechanism

The framework includes a small warmup mechanism that executes several trivial solver runs before the actual fuzzing experiment begins. The purpose of this step is to avoid measuring one-time startup overheads such as Python imports and DSL toolchain startup.

This mechanism was introduced after encountering issues with the Gnark backend, where the Go toolchain attempted to build parts of the project during the first execution. However, the build itself exceeded the experiment timeout, causing the iteration to terminate before the build completed. Since the partially completed build artifacts were discarded, the next iteration restarted the same build from scratch. In

practice, this meant that the entire timeout budget of every iteration was consumed by repeated incomplete builds, preventing the experiment from making meaningful progress.

To mitigate this, the framework first executes several minimal SAT and UNSAT benchmarks through the normal pipeline and discards the results. This ensures that later measurements more accurately reflect the cost of the actual testing workflow rather than transient initialization behaviour.

4.5.4 Technical Stack

The framework was implemented primarily in Python with some DSL-specific parts written in Go and Rust. SMT formulas were parsed and manipulated using `pySMT` [64], while Z3 [46] and CVC5 [47, 62] were used as the main SMT backends during benchmark generation and oracle execution. The fuzzing component of the framework was based on YinYang [9]. Constrainedness analysis was performed using Picus [19, 18]

For test-case minimisation when developing and debugging the tool and underlying compilers, I additionally used ddSMT [50].

4.5.5 Results

Multiple fuzzing campaigns were conducted using the framework described in this chapter. Although these experiments did not uncover any logical constraint-generation bugs in the evaluated ZKP compilers, they did result in the discovery of two bugs in the finite-field backend of CVC5. Both issues are covered in Section 5.3.

Since no real-world constraint-generation bugs were identified during these campaigns, the effectiveness of the framework is evaluated in Chapter 5 using alternative methodologies, namely compiler coverage analysis and artificial bug-injection experiments.

4.6 A Late-Stage Exploration of Noir and ACIR

This section describes a differential testing framework for the Noir compiler and its Barretenberg backend that was developed as a late-stage extension of the main project. This work was undertaken very late in the project, after the design, implementation, and evaluation (Chapter 5) of the main SMT-based framework had already been completed.

Although the SMT-based framework from this chapter proved effective at reasoning about the correctness of generated constraint systems, it was primarily focused on a single class of bugs: logical errors introduced during constraint generation. While this focus was deliberate, it didn't manage to uncover any of such bugs in the wild.

Rather than further refining the existing framework, I decided to use the remaining project time to explore a broader testing strategy. Noir was chosen as the target of this investigation because the bug analysis in Section 5.2 suggested that it contained a larger set of bugs than the other evaluated systems. Furthermore, Noir operates at a higher level of abstraction than other DSLs, providing features such as fixed-width integers, automatic type conversions, and array operations.

A further motivation came from the difficulties encountered when experimenting with integer SMT theories earlier in this chapter. Translating integer operations into R1CS-based systems typically requires large bit-decomposition circuits, making the resulting constraint systems difficult to analyse. Because Noir exposes integer operations directly, it provides a more natural target for SMT-based testing of integer and array theories.

The resulting framework reuses ideas developed throughout this section, including SMT-based generation and constraint-level analysis, but broadens the scope of testing.

4.6.1 Architecture Overview

The overall design combines ideas from both the SMT-based framework presented earlier in this chapter and the end-to-end proving oracle used by CIRCUZZ. The aim is to exercise a wider portion of the Noir toolchain, including ACIR generation, witness generation, and the Barretenberg proving backend.

Benchmarks are generated using SMTFUZZ [54, 55], which produces random SMT formulas from integer and array theories. Unlike the earlier framework, this approach does not use fusion-based metamorphic transformations. Since this investigation was intended as an exploration of a different testing direction, I deliberately chose a different benchmark generator rather than extending the existing YINYANG-based pipeline. The two approaches are not mutually exclusive and fusion could be integrated into the framework in future work. Furthermore, SMTFUZZ does not currently support finite-field theories, which made it unsuitable for the earlier SMT-based framework that focused heavily on finite-field benchmarks. The generated formula is solved directly using an SMT solver and the resulting satisfiability result serves as the expected behaviour.

The framework currently supports formulas from the QF_LIA, QF_NIA and QF_ANIA logics. Since SMT integers are unbounded while Noir uses fixed-width machine integers, the generated formulas are augmented with additional constraints before solving. These constraints ensure that generated witnesses remain compatible with the selected Noir types while simultaneously exercising the language's casting and integer semantics.

For satisfiable formulas, the SMT model is translated into a Noir witness and executed through the standard Noir toolchain. The witness is then additionally mutated and rechecked using the SMT solver until a modified assignment that no longer satisfies the formula is obtained. The framework then verifies that the corresponding Noir execution also fails. For unsatisfiable formulas, a random witness is generated and execution is expected to fail. Successful executions are additionally passed through the Barretenberg proving pipeline.

Finally, the framework includes an ACIR-level SMT oracle similar to the one described earlier in this chapter, allowing the compiled circuit to be analysed independently of the original source program. Unlike the earlier framework, only SAT and UNSAT oracles were implemented. Although integrating the Picus-based constrainedness oracle would be possible, I didn't have enough time to implement this extension.

4.6.2 SMT-to-Noir Translation

Integer Theories

The main challenge in translating Integer SMT formulas into Noir programs is the mismatch between the underlying integer models. SMT integer theories operate over unbounded mathematical integers, whereas Noir provides only fixed-width machine integer types such as `i8`, `i16`, `u32`, and `i64`. Consequently, a direct translation is not possible.

To address this issue, each SMT variable is assigned a concrete Noir integer type. Types are selected randomly from a configurable pool, subject to simple constraints that ensure the chosen type can represent the constants appearing in the corresponding SMT expressions. Types are then propagated through the expression tree. Whenever an operation combines values of different types, explicit casts are inserted according to Noir's promotion rules.

For example, consider the SMT expression

```
(assert (> (+ x y) 10))
```

where (x) has been assigned type `u16` and (y) type `i32`. Since the operands differ in both width and signedness, the expression is promoted to a common type capable of representing both values. In this case, `i32` is selected and the unsigned operand is explicitly cast before the operation is performed:

```
let tmp = (x as i32) + y;
assert(tmp > 10);
```

To ensure that SMT-generated witnesses remain valid after translation, the original formula is augmented with additional range constraints before solving. The framework assigns types not only to variables but also to every integer subexpression. Additional SMT constraints are then introduced to ensure that the value of each subexpression remains representable in its assigned type.

Since unsigned values may later be promoted to signed types during expression evaluation, the framework conservatively uses the upper bound of the corresponding signed type when generating range constraints. For example, although `u16` normally permits values up to (65535), variables of this type are restricted to at most (32767). This avoids situations where a value is representable in its original type but overflows after promotion to a signed type in a larger expression.

For example, consider the SMT formula

```
(declare-fun x () Int)
(declare-fun y () Int)

(assert (> (+ x y) 10))
```

Assume that (x) is assigned type `u16` and (y) type `i32`. The expression (x + y) is assigned type `i32`, so the framework constrains both the variables and the result of the addition:

```
(declare-fun x () Int)
(declare-fun y () Int)

(assert (<= -32768 x))
(assert (<= x 32767))

(assert (<= -2147483648 y))
(assert (<= y 2147483647))

(assert (<= -2147483648 (+ x y)))
(assert (<= (+ x y) 2147483647))

(assert (> (+ x y) 10))
```

This approach is not perfect. The bound inference mechanism does not fully model every implicit conversion and casting rule used by Noir. Consequently, some generated witnesses may satisfy the augmented SMT formula while still triggering an overflow or cast-related failure after translation. Such cases result in false positives and require manual inspection.

Array Theory

Supporting SMT array theories introduces an additional challenge: SMT arrays are conceptually unbounded, whereas Noir arrays have a fixed compile-time size. Consequently, array sizes must be inferred during translation.

The framework assigns each SMT array type a concrete Noir array size based on the largest constant index access appearing in the formula. For example, if the formula contains an access of the form

```
(select arr 17)
```

then the generated Noir array must contain at least 18 elements. To ensure that generated witnesses remain valid, every expression used as an array index is additionally constrained to lie within the valid index range before SMT solving.

To simplify translation, all arrays of the same SMT type are assigned the same size. For example, all arrays of type

```
(Array Int Int)
```

within a benchmark are translated into Noir arrays of identical size. This allows array equalities to be translated directly, since Noir requires both arrays to have the same length.

Array formulas introduce additional complications with respect to typing. Unlike the integer-only benchmarks, array benchmarks use only unsigned integer types (`u8`, `u16`, and `u32`). This simplifies array index handling, since Noir expects array accesses to use unsigned integer indices. Supporting the full signed and unsigned integer hierarchy together with the required index conversions would complicate the translation process.

Finally, the current implementation does not support arrays indexed by arrays, such as

```
(Array (Array Int Int) Int)
```

or similar nested constructions. Such benchmarks are discarded during generation. Although these formulas are valid SMT, they do not map naturally to Noir's array model and are not encountered often enough in SMTFUZZ to meaningfully affect the throughput.

4.6.3 ACIR-to-SMT Translation

The ACIR SMT oracle is conceptually similar to the constraint-level SMT oracle described earlier in this chapter. In both cases, the compiled constraint system is translated back into SMT and solved independently of the original source program. However, ACIR exposes additional constructs that do not appear in the R1CS representation used by the earlier framework.

Arithmetic assertions are translated in the same way as R1CS was translated previously. The oracle additionally supports `BLACKBOX::RANGE` constraints, which are translated into SMT bounds on the corresponding witness variables.

ACIR oracle also supports memory operations. The model was divided into three conceptual operations: initialisation (`INIT`), reads (`READ`), and writes (`WRITE`). These correspond to the ACIR `MemoryInit` and `MemoryOp` opcodes and were translated into SMT arrays as follows.

- **INIT**
  The NOIR code

  ```
  let arr = [10, 20, 30];
  ```

  produces an ACIR memory initialisation operation conceptually equivalent to

  ```
  MemoryInit(arr, [10,20,30])
  ```

  which is translated into SMT as

  ```
  (assert (= (select arr 0) 10))
  (assert (= (select arr 1) 20))
  (assert (= (select arr 2) 30))
  ```

  The SMT array variable `arr` therefore represents the entire ACIR memory block.
- **READ**
  The NOIR code

  ```
  let x = arr[i];
  ```

  where *i* is not constant statically generates an ACIR memory read operation conceptually equivalent to

```
MemoryOp(arr, i, READ)
```

and is translated into SMT as

```
(assert (= x (select arr i)))
```

Dynamic indexing is one of the main situations where Noir emits memory operations rather than compiling the access away statically, since dynamic array accesses are non-trivial to represent in ZKP constraint systems.

- **WRITE**
  The Noir code

```
arr[i] = v;
```

generates an ACIR memory write operation conceptually equivalent to

```
MemoryOp(arr, i, WRITE, v)
```

which is translated into SMT using the `store` operator:

```
(assert (= arr_1 (store arr_0 i v)))
```

Since SMT arrays are immutable, each write creates a fresh array version. Sequential updates are therefore represented as

```
(assert (= arr_1 (store arr_0 i v)))
(assert (= arr_2 (store arr_1 j w)))
```

4.7 Summary

This chapter presented an SMT-based testing framework for ZKP compilers. The framework combines SMT-based benchmark generation, automated translation into multiple DSLs, and constraint-level analysis using satisfiability and constrainedness oracles. To support these goals, I introduced benchmark generation techniques based on YinYang, SMT-based translations for multiple constraint-system representations, and several practical optimisations required to make the approach scalable.

The chapter also described a later exploratory extension targeting Noir and its ACIR representation. While this work was not part of the main evaluation, it demonstrates how the core ideas developed throughout the project can be applied to higher-level compiler abstractions and alternative testing strategies.

The effectiveness of the framework is evaluated in Chapter 5, where I analyse compiler coverage, artificial bug-injection experiments, and the bugs uncovered during the project.

Chapter 5

Evaluation

This chapter evaluates the techniques introduced in the previous two chapters, with a particular focus on the SMT-based testing framework. While the CIRCUZZ improvements presented earlier provide important context and are discussed where relevant, the primary goal of this chapter is to assess the effectiveness of the SMT-based approach. The Noir extension described at the end of Chapter 4 is not included in the evaluation, as it was developed after the main experimental campaign had already been completed.

The evaluation studies compiler coverage, artificial bug injection, throughput scalability with benchmark size, and the impact of adaptive hint injection on oracle performance. It also compares finite-field and Boolean benchmarks in terms of diversity and practical testing behaviour.

In addition, I include a historical analysis of public bug reports in selected ZKP compiler repositories. This analysis provides additional context for how logical constraint-generation bugs have appeared in practice, and helps explain why the main experimental campaign may not have uncovered such bugs in the wild.

Finally, the chapter presents the bugs discovered during the project. Although no logical constraint-generation bugs were found in the evaluated compilers, during this project I still uncovered several other issues, including compiler crashes, backend inconsistencies, and solver failures.

The evaluation is structured around two primary research questions and several supporting research questions.

Primary Research Questions

- **RQ1:** Can the SMT-based approach effectively exercise ZKP compiler compilation and optimisation pipelines?
- **RQ2:** Can the SMT-based framework efficiently detect logical bugs in generated constraint systems?

Supporting Research Questions

- **RQ3:** How does benchmark size affect testing throughput and oracle performance of the SMT-based framework?
- **RQ4:** Does adaptive hint injection improve SMT-solving performance during oracle execution?
- **RQ5:** Do finite-field benchmarks offer practical advantages over Boolean benchmarks for SMT-based testing?

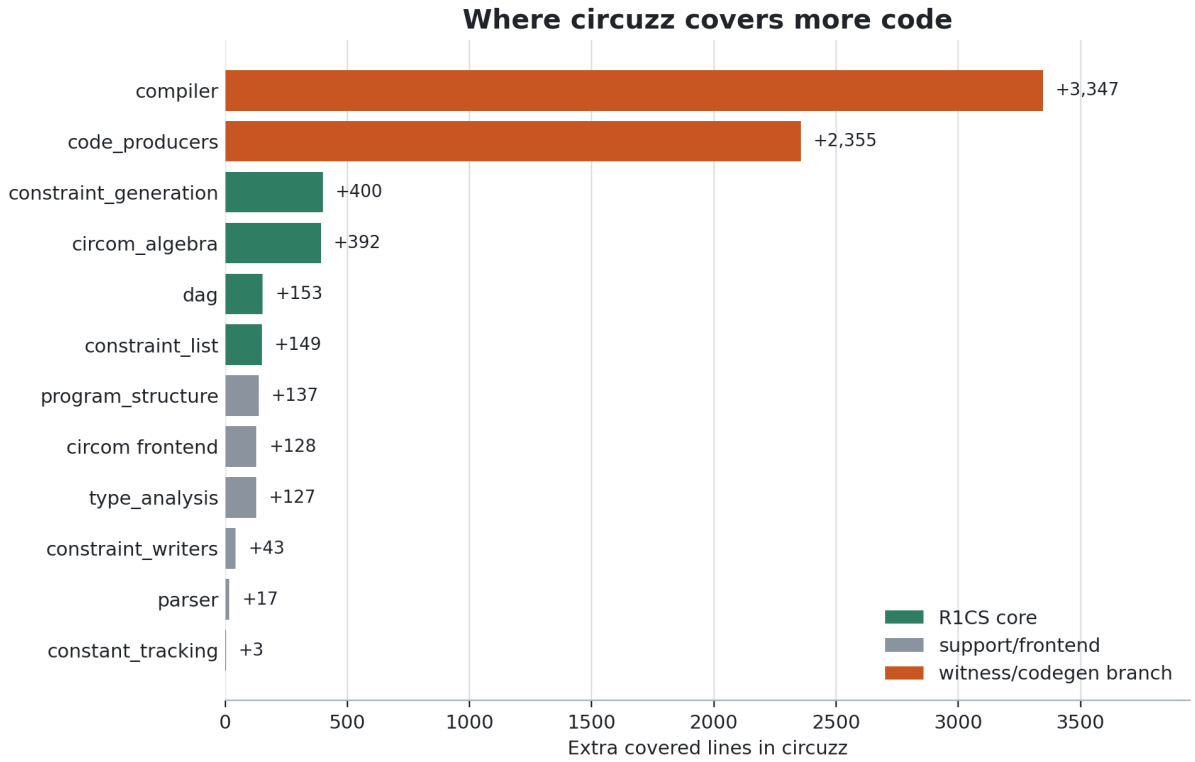

Figure 5.1: Compiler components where the fully-constrained Circuzz workload covered more lines than the SMT-based approach.

5.1 Experimental Evaluation

The reports, raw experimental results, and scripts used for the evaluation can be found in the `evaluation/` directory of the accompanying code repository.

5.1.1 Compiler Coverage

To evaluate whether the SMT-based approach is capable of exercising realistic compiler behaviour, I conducted a differential coverage experiment against the fully-constrained Circuzz Circom generator introduced in Section 3.2.1. Rather than measuring raw compiler coverage in isolation, I used differential coverage, since it provides essentially the same information while additionally enabling a direct comparison against an existing testing tool. The experiment was performed on the Circom compiler using LLVM source coverage instrumentation [65]. Both tools were executed for 15 minutes under the same coverage configuration. The SMT-based configuration used the Boolean-based benchmark generator. A full line-by-line coverage breakdown is provided in Appendix D. Figure 5.1 summarises the compiler components where Circuzz achieved higher coverage. Table D.1 reports the corresponding line-count differences for the main affected modules.

The results show that the fully-constrained Circuzz workload achieved higher overall compiler coverage, covering 16,443 lines compared to 9,192 lines for the SMT-based approach. However, most of this difference originates from later witness-generation and export stages of the compiler, which are not directly relevant to constraint generation and are, by design, largely not exercised by the SMT-based tool. When restricting the analysis only to the parts of the compiler directly involved in constraint generation and optimisation, the gap becomes significantly smaller, decreasing to only 1,097 lines. If code related to `assert()` witness hints is additionally excluded, the difference drops further to roughly 300 lines. Despite the lower overall coverage, the SMT-based approach successfully exercised all major stages of the R1CS compilation pipeline. Most importantly, the generated benchmarks were able to trigger the primary optimisation pipeline, including iterative simplification and substitution

Module	Circuzz	Picus	Delta	Gap	Relevant
`constraint_generation`	2636	2236	+400	15.2%	Yes
`constraint_list`	819	670	+149	18.2%	Yes
`circom_algebra`	1183	791	+392	33.1%	Yes
`dag`	555	402	+153	27.6%	Yes
`constant_tracking`	21	18	+3	14.3%	Yes
`constraint_writers`	294	251	+43	14.6%	No
`parser`	536	529	+7	1.3%	No
`type_analysis`	1750	1623	+127	7.3%	No
`program_structure`	1565	1428	+137	8.8%	No
`circom` frontend	661	533	+128	19.4%	No
`compiler` all	4058	711	+3347	82.5%	No
`code_producers`	2355	0	+2355	100%	No

Table 5.1: Module-level coverage differences between `circuzz-arithmetic` and `picus-fusion`. The *Relevant* column marks modules directly related to constraint generation and optimisation.

handling. The analysis also revealed two important observations. First, neither workload triggered the newer `substitution_process_4()` optimisation path, instead remaining on the older `process_3` simplification pipeline. Second, the SMT-based benchmarks appeared unable to trigger more than one round of optimisation passes. These observations, together with possible explanations and future improvements, were already discussed in Section 4.5. See also Appendix A, which contains an in-depth analysis and detailed interpretation of the Circom results.

RQ1 Overall, the experiment provides a positive answer to **RQ1**. The SMT-based approach was able to exercise the core compilation and optimisation pipeline of the Circom compiler. Although it did not reach as much of the surrounding witness-generation infrastructure, this is expected given the design and goals of the tool. While the experiment was conducted only on Circom, the conclusion should generalise to other ZKP DSLs, since most such compilers ultimately implement similar constraint-generation and optimisation pipelines over arithmetic constraint systems.

5.1.2 Artificial Bug Detection

Although the framework discovered several other issues, no real-world logical constraint-generation bugs were found in the evaluated compilers. This makes evaluation difficult, since the main class of bugs targeted by the framework appears to be relatively rare (see Section 5.2). Artificial bug injection was therefore used to create a controlled setting in which the ability of the framework to detect semantic inconsistencies could be evaluated directly.

To evaluate **RQ2**, I conducted a series of artificial bug injection experiments on the Circom compiler. The goal was to measure whether the framework can reliably detect logical inconsistencies introduced directly into the generated constraint systems. The experiments were performed by modifying the compiler to inject an artificial logical bug into every compilation. More precisely, the modified compiler selected a random non-linear constraint and perturbed it by adding a small constant delta chosen from:

$$\{-3, -2, -1, 1, 2, 3\}.$$

This preserves the overall structure of the generated constraint system while introducing an incorrect relation between variables. Each experiment consisted of 5 independent fuzzing instances executed with a 5-minute timeout budget. The SMT configurations used Boolean-based benchmarks together with adaptive hint injection enabled. Table 5.2 shows aggregated results across all 5 fuzzing instances. More in-depth per-iteration results can be found in Appendix E. For each experiment, the number of successful runs, generated programs, and total execution time were aggregated across all 5 runs and used to compute the following metrics:

- **Bugs Found.**
  Number of fuzzing instances (out of 5) that discovered a bug.
- **Bugs / Program.**
  Total number of discovered bugs divided by the total number of generated programs across all runs.
- **Bugs / Second.**
  Total number of discovered bugs divided by the total execution time summed across all runs.

Configuration	Bugs Found	Bugs / Program	Bugs / Second
SMT SAT	5/5	0.2273	0.0166
SMT + Picus	5/5	0.2273	0.0104
SMT UNSAT	0/5	0.0000	0.0000
Circuzz Standard	2/5	0.0088	0.0014
Circuzz + Picus	0/5	0.0000	0.0000

Table 5.2: Aggregate results of the artificial bug injection experiments.

Several important observations can be drawn from these results:

- The SMT-based SAT and Picus modes significantly outperformed both Circuzz configurations. Although Circuzz Standard was still able to discover some bugs, it is important to note that the injected bug was present in every generated program. The SMT-based approach therefore achieved substantially higher bug-detection efficiency, answering **RQ2** positively.
- The UNSAT oracle performed extremely poorly, failing to detect any injected bugs in any run. This suggests that unsatisfiability-preserving mutations may be substantially harder to violate than constrainedness-based properties. Another possible explanation is insufficient benchmark quality. This requires further investigation.
- The combination of Circuzz with the Picus oracle performed particularly poorly. Additional throughput measurements indicate that this is primarily caused by low execution throughput, with the configuration achieving only approximately 0.03 programs per second. This suggests that randomly generated Circuzz circuits are not well suited for expensive constrainedness-based analysis with Picus.
- The SMT-based approach also exhibits substantially lower throughput than Circuzz, which is expected given the significantly more involved SMT oracle and solver interaction. This can be observed by comparing the relative improvements in *Bugs / Program* and *Bugs / Second*. While the SMT approach achieves dramatically higher bug-detection efficiency per generated program, the improvement in bugs discovered per second is noticeably smaller due to the higher computational cost of each test case.

5.1.3 Benchmark Size and Adaptive Hints

To jointly evaluate **RQ3** and **RQ4**, I measured mean solve time as a function of benchmark size, comparing configurations with and without adaptive hint injection. Benchmarks were constructed by applying YinYang semantic fusion [9] to Boolean SAT seed formulas pruned to n variables, producing fused formulas with between $2n+1$ and $3n$ Boolean variables. Five benchmarks were generated per value of n, sweeping n from 1 to 14.

All benchmarks were solved using the Circom backend with the cvc5 solver and a per-benchmark timeout of 120 seconds. Two configurations were evaluated in parallel: one with adaptive hint injection enabled and one disabled. Five benchmarks were evaluated per value of n. The run was stopped automatically once all five benchmarks at a given n timed out or returned `unknown`, as further increases in n were unlikely to produce any solvable instances.

The results are shown in Figure 5.2. Both configurations exhibit approximately exponential growth in mean solve time as n increases. At small values of n, both configurations perform comparably, but the

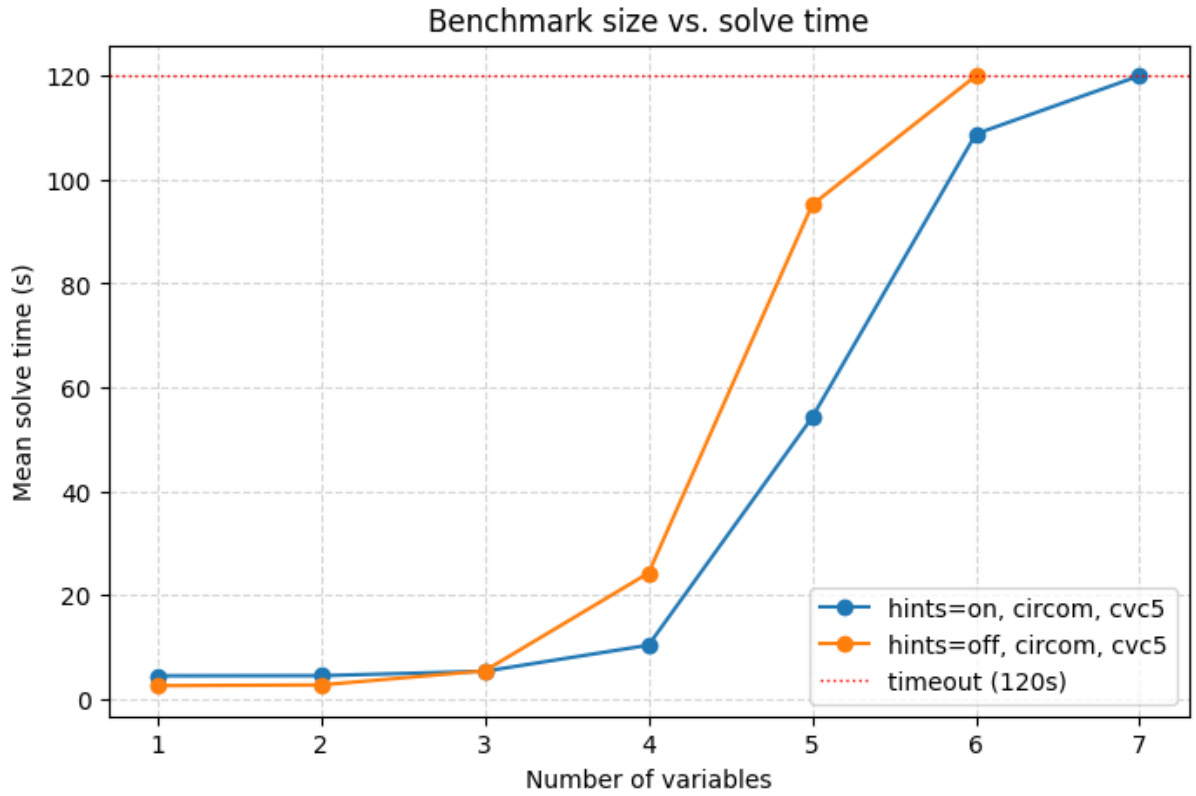

Figure 5.2: Mean solve time versus benchmark size (number of variables n in the seed formula) for runs with adaptive hints (hints=on) and without them (hints=off). Timeouts are counted at the timeout value of 120 seconds.

divergence between hints=on and hints=off becomes clearly visible at higher values of n, where hints=on achieves lower solve times and tolerates one additional step before reaching the 100% timeout threshold. In terms of scalability, hints=off reaches 100% timeout at $n = 6$, whereas hints=on continues to solve some benchmarks at $n = 6$ (40% timeout rate) before hitting 100% timeout at $n = 7$.

RQ3 Mean solve time grows approximately exponentially with benchmark size.

RQ4 Adaptive hint injection provides a performance benefit at larger benchmark sizes. The improvement is negligible at small n, where the hint-generation overhead is not recovered on easy instances, but becomes substantial as n grows.

5.1.4 Boolean vs Finite-Field Benchmarks

To evaluate **RQ5**, I conducted another differential coverage experiment comparing the Circom compiler coverage achieved when testing with Boolean benchmarks against finite-field benchmarks. Both experiments ran for 15 minutes under identical configurations using YinYang semantic fusion. The results show that the two encodings cover nearly identical parts of the compiler. Of 9,292 total covered lines, 9,185 (98.8%) are shared between both configurations. The Boolean encoding uniquely covers 92 lines and the finite-field encoding uniquely covers 15 lines. The differences are confined to a small number of paths in `algebra.rs`, which is the compiler's arithmetic IR layer responsible for building the R1CS expression representation from source code, rather than the constraint optimisation pipeline. All remaining compiler subsystems - in particular the constraint optimisation pipeline - are exercised identically by both encodings.

RQ5 Switching from Boolean to finite-field benchmarks does not meaningfully change which parts of the Circom compiler are exercised. The two encodings explore over 98% of the same compiler paths,

and the small remaining differences are limited to specific arithmetic operator compilation paths. Finite-field benchmarks do not offer a practical coverage advantage over Boolean benchmarks for SMT-based compiler testing.

5.2 Historical Bug Analysis

Repository	Scraped	Relevant
`iden3/circom`	9	3
`ConsenSys/gnark`	139	14
`Zokrates/ZoKrates`	71	3
`noir-lang/noir`	357	87
Total	**576**	**107**

Table 5.3: Issues matching keywords and classified as relevant bug reports by this process.

To provide additional context for the experimental results, I conducted an exploratory analysis of publicly reported issues in four major ZKP compiler repositories: `iden3/circom`, `ConsenSys/gnark`, `Zokrates/ZoKrates`, and `noir-lang/noir`. The motivation for this analysis was to better understand how logical constraint-generation bugs have appeared in practice, and to help interpret why the main experimental campaign did not uncover such bugs in the evaluated compilers.

This analysis is based on keyword searches in public issue trackers and LLM-assisted classification, so the results are only approximate. In particular, it may miss relevant issues that use different terminology and include irrelevant issues that match the selected keywords. Nevertheless, it provides useful supporting context for understanding the kinds of logical issues that have historically been reported in ZKP compiler implementations.

Issues were collected using a custom scraper that queried the GitHub Search API using a predefined set of keywords, including terms such as *underconstrained*, *missing constraint*, *soundness bug*, and *overflow*. An issue is counted as *scraped* if it matched at least one of these keywords. The scraped results were then reviewed by an LLM, which classified each issue as either *relevant* or noise.

The results are shown in Table 5.3. For Circom and ZoKrates, the number of issues classified as relevant is very small. In the case of Circom, two of the three relevant issues were filed in 2026 and are closely related: issue `#406` reports silent field reduction of runtime inputs, and issue `#407` is a combined report of ten bugs found by differential testing. Of the ten sub-bugs reported in `#407`, only one concerns the generated constraint system directly: compile-time constants exceeding the field prime being silently reduced in the generated R1CS. The remaining nine sub-bugs concern the witness generator rather than the constraint system. The third relevant issue, `#371`, reports three under-constrained circuits in the Circomlib standard library.

For Gnark, 14 issues were classified as relevant. Of these, 5 concern bugs in Gnark's standard cryptographic circuit library (`std/hash`, `std/signature`), where circuits implementing pairing checks, hash functions, and elliptic curve arithmetic were found to be under-constrained or unsound. These issues are important, but they are closer to bugs in hand-written library circuits than to general compiler constraint-generation bugs. The remaining 9 issues concern the compiler and field emulation layer more directly.

Noir accounts for the large majority of issues classified as relevant, with 87 reports in total. This is consistent with Noir being a significantly higher-level language with complex constructs such as generics, arrays, closures and casting, where incorrect constraint generation may arise in more varied ways.

Overall, this analysis suggests that logical constraint-generation bugs do occur across ZKP compilers, but that they appear relatively uncommon in the lower-level languages directly targeted by the main evaluation: Circom, ZoKrates, and Gnark. This helps explain why the main experimental campaign

did not uncover real-world logical constraint-generation bugs in those compilers, despite finding crashes, backend inconsistencies, and solver failures. The picture is different for Noir, where the number of reports classified as relevant is substantially higher. However, many of these reports involve advanced language features such as arrays, casting, and Brillig operations that were not exercised by the SMT-based testing approach evaluated in the main experiments.

5.3 Bugs Found

Although no errors that can obviously be classified as logical bugs in constraint generation were found during the main evaluation, the project uncovered several additional bugs in ZKP compilers and SMT solvers. Furthermore, late-stage fuzzing campaigns targeting Noir (see Section 4.6) uncovered additional issues that are more closely related to the original objective. The rest of this section describes each issue.

ID	System	Bug Type	Status
B1	`snarkjs`	PLONK backend failure	Reported
B2	`snarkjs`	Incorrect error condition	Reported
B3	`ZoKrates`	Compiler panic (`unreachable`)	Reported
B4	`ZoKrates`	Compiler panic (`unimplemented`)	Reported
B5	`cvc5`	Finite-field backend crash	Fixed upstream
B6	`cvc5`	Gröbner basis tracer crash	Known
B7	`Noir`	Misleading casts of negative integers	Warning added upstream
B8	`Noir`	Unsigned variable produces `-1` array index	Reported

Table 5.4: Summary of bugs discovered during the project.

B1: `snarkjs` PLONK backend fails with no output signals

Issue: `https://github.com/iden3/snarkjs/issues/617`
Triggered by: Circuzz with the fully-constrained Circom generator

The following circuit causes `snarkjs plonk prove` to fail with `Evaluations.getEvaluation() out of bounds`, despite successful circuit compilation, witness generation, witness checking, and PLONK setup:

```
template main_template() {
    signal input in0;
    (in0 - 15) * (in0 - 16) === 0;
}

component main = main_template();
```

Inspecting the generated `.zkey` file shows that Section 3 has size zero. Adding a dummy output signal avoids the proving failure.

B2: `snarkjs` incorrectly rejects $0 > 2^{17}$

Issue: `https://github.com/iden3/snarkjs/issues/624`
Triggered by: Circuzz with the fully-constrained Circom generator

`snarkjs` raises an error during PLONK key generation for some circuits with zero constraints, reporting:

```
circuit too big for this power of tau ceremony.  0 > 2^17
```

The condition is trivially false, since $0 \not> 2^{17}$.

B3: ZoKrates panic due to unreachable code

Issue: https://github.com/Zokrates/ZoKrates/issues/1390
Triggered by: Circuzz with the ZoKrates backend

The following minimized program causes the compiler to panic:

```
def main(private field in0) -> field {
    field cond0 = if (((-1f) * in0 * (-1f)) > in0) { 1f } else { 0f };
    return cond0;
}
```

The compiler terminates with an internal `entered unreachable code` panic.

B4: ZoKrates panic due to unimplemented comparison

Issue: https://github.com/Zokrates/ZoKrates/issues/1391
Triggered by: Circuzz with the ZoKrates backend

The following minimized program causes the compiler to panic:

```
def main(private field in0) -> field {
    assert((in0 * in0) > (-3f));
    return 0f;
}
```

The compiler crashes with a `not implemented` panic related to comparisons involving negative constants.

B5: cvc5 crash with -ff-solver=split

Issue: https://github.com/cvc5/cvc5/issues/12627
Triggered by: SMT SAT oracle with ET-generated finite-field benchmarks

The following benchmark causes `cvc5` to terminate with a `CoCoA::ErrorInfo` runtime exception:

```
(set-logic QF_FF)
(declare-const a (_ FiniteField 3))
(declare-const b (_ FiniteField 3))
(assert (= (ff.mul a b) (ff.mul b a)))
(check-sat)
```

The issue was later fixed upstream.

B6: cvc5 finite-field Gröbner basis tracer crash

Issue: https://github.com/cvc5/cvc5/issues/10937
Triggered by: SMT UNSAT oracle with Poseidon-based finite-field benchmarks

The following benchmark triggers a crash in older `cvc5` versions:

```
(set-logic QF_FF)
(declare-const x (_ FiniteField 7))
(assert (= (ff.mul x x) x))
(check-sat)
```

The issue was caused by inconsistencies between internally normalised polynomials and the Gröbner basis tracer state. It has since been fixed in newer `cvc5` versions.

B7: `Noir` misleading casts of negative integers

Issue: `https://github.com/noir-lang/noir/issues/12794`
Triggered by: Late-stage SMT-based fuzzing of Noir; ACIR SMT oracle

The ACIR SMT oracle uncovered an inconsistency in Noir's handling of negative integer literals combined with casts. Expressions such as `(-1 as i8)` were interpreted differently from the semantically equivalent `-(1 as i8)`, causing assertions that should hold to fail. The issue originated from negative literals first being interpreted as field elements before the cast was applied. Noir developers decided that the underlying behaviour was consistent with the language's type system and therefore did not change its semantics; however, they agreed that the syntax was highly misleading and error-prone. As a result, a dedicated compiler warning was added upstream to alert users when this pattern occurs.

B8: `Noir` unsigned variable produces `-1` array index

Issue: `https://github.com/noir-lang/noir/issues/12841`
Triggered by: Late-stage SMT-based fuzzing of Noir; witness generation oracle

The framework discovered a program in which Noir reported an unsigned integer value as negative. The array access failed with an out-of-bounds error stating that the index was `-1`. The original issue has since been closed, but the discussion was moved to a separate issue.

5.3.1 Compiler Dependence on System Time

During experiments, I encountered an unusual crash in the Circom compiler caused by dependence on wall-clock time during compilation. I am still unsure whether this should be considered a compiler bug, since the failure requires artificial manipulation of the system clock to trigger, so I am presenting it separately as an interesting case. The issue was reproduced using `libfaketime` [66] (a tool for mocking the system clock) while compiling circuits. Under certain clock adjustments, the compiler panicked with a `SystemTimeError` originating from the constraint simplification phase. The crash occurs in timing and profiling code rather than in the actual simplification algorithm itself. The relevant code measures elapsed wall-clock time using `SystemTime::now()` followed by `elapsed().unwrap()`. `elapsed()` can fail if the system clock moves backwards, for example, during container or WSL clock adjustments. The use of `unwrap()` then converts this recoverable condition into a hard compiler crash. The same unsafe timing pattern appears multiple times throughout the compiler. To better understand whether this behaviour was unusual, I performed similar experiments on C, Go, and Rust compilers while using `libfaketime`. None of them exhibited comparable failures. This issue raises an interesting question about whether compiler execution should depend on wall clock behaviour at all.

Chapter 6

Conclusion and Further Work

6.1 Summary and Reflection

This project explored the problem of testing logical correctness in ZKP compilers, with a particular focus on semantic bugs in constraint generation. The project initially began by experimenting with extensions to existing ZKP compiler fuzzers. During this process, I identified logical constraint-generation bugs as a relatively underexplored area that existing tools could struggle to detect.

To investigate this problem, I designed and implemented an SMT-based testing framework capable of generating benchmark programs, compiling them into ZKP constraint systems, and reasoning about their semantics using external oracles. The framework combined SMT-based program generation with solver-based analysis of the resulting constraints.

Although the framework uncovered several compiler crashes, backend failures, and solver inconsistencies, it did not discover any real-world logical constraint-generation bugs in the evaluated compilers. This makes it unclear whether the framework was ineffective at finding such bugs or whether these bugs are relatively uncommon in mature ZKP compilers.

To address this, I conducted artificial bug injection experiments by modifying the Circom compiler to introduce semantic inconsistencies directly into generated constraints. In this setting, the SMT-based framework consistently detected injected bugs. The existing Circuzz framework was also capable of finding such bugs, although significantly more slowly. Further evaluations additionally examined compiler coverage, constrainedness analysis, and finite-field benchmark generation.

Despite not finding real-world logical bugs in the evaluated ZKP compilers, the project still produced several reusable contributions. These included new finite-field benchmark suites for SMT solvers, practical exploration of finite-field SMT solving, and optimisation techniques for improving oracle performance. In particular, adaptive hint injection improved SMT solver scalability on large constraint systems.

Very late in the project, I also extended the framework in a different direction by investigating SMT-based differential testing of the Noir compiler and its ACIR representation. Rather than focusing exclusively on constraint-generation correctness, this extension explored a broader class of semantic compiler behaviours, including integer arithmetic, type conversions, array operations, and ACIR-specific constructs. Exploratory fuzzing campaigns uncovered two additional bugs in the Noir compiler. While neither bug was a direct constraint-generation error, both exhibited a similar semantic flavour and resulted in incorrect compiler behaviour. This suggests that broadening the scope of SMT-based testing may be a promising direction for future research.

The historical bug analysis additionally suggested that the original choice of target DSL may not have been ideal. Languages such as Circom compile into relatively simple low-level arithmetic constraints, limiting the space for semantic compiler mistakes. In contrast, newer DSLs such as Noir, with higher-level

abstractions, richer type systems, and unconstrained execution features, appear to provide a substantially larger attack surface for semantic testing.

Overall, the project demonstrated that SMT-based semantic testing of ZKP compilers is feasible and capable of detecting logical inconsistencies when they are present. The controlled bug-injection experiments showed that the approach is effective at detecting semantic faults in constraint generation, while the later Noir investigation demonstrated that many of the same ideas can also be applied to broader compiler correctness problems. Together, these results highlight the potential of SMT-based techniques as a practical tool for testing the correctness of ZKP tooling and suggest several promising directions for future work.

6.2 Further Work

Several directions for future work emerged during this project. The following list highlights the most interesting and promising ones.

- **Focus more heavily on Noir and ACIR.**
  The exploratory Noir extension produced some of the most promising results of the project. Compared to low-level DSLs such as Circom, Noir contains richer language features, more complex compiler transformations, and a larger surface for semantic bugs. Future work could therefore benefit greatly from focusing on Noir.
- **Explore additional SMT theories.**
  Future work could investigate using different SMT theories for program generation. For example, Noir has recently introduced experimental support for string processing and regular-expression matching through projects such as zkRegexp [25], providing a natural target for future SMT-based testing efforts using the String-based theories (e.g QF_SLIA).
- **Broaden the scope of testing objectives.**
  The original framework was designed specifically to detect logical constraint-generation bugs. The later Noir work suggests that the same SMT-based generation techniques can be applied to a much broader range of compiler correctness problems. Future work should therefore explore a wider variety of bug classes rather than focusing on a single category.
- **Investigate additional semantic testing oracles.**
  The framework currently relies on satisfiability, unsatisfiability, and constrainedness-based reasoning. Future work could integrate additional tools such as zkFuzz, allowing the same benchmarks to be analysed using multiple complementary semantic testing approaches.

References

[1] Ozdemir A, Wahby RS, Brown F, Barrett CW. Bounded Verification for Finite-Field-Blasting in a Compiler for Zero Knowledge Proofs. In: Computer Aided Verification. Springer; 2023. p. 154-75. Accessed: 2026-01-18. Available from: https://doi.org/10.1007/978-3-031-37709-9_8.

[2] Ernstberger J, Chaliasos S, Zhou L, Jovanovic P, Gervais A. Do You Need a Zero-Knowledge Proof?; 2024. Accessed: 2026-01-18. Cryptology ePrint Archive, Paper 2024/050. Available from: https://eprint.iacr.org/2024/050.

[3] Coinbase. Zcash (ZEC) Price; 2026. Accessed: 2026-01-18. Available from: https://www.coinbase.com/price/zcash.

[4] Bellés-Muñoz M, Isabel M, Muñoz-Tapia JL, Rubio A, Baylina Melé J. Circom: A Circuit Description Language for Building Zero-Knowledge Applications. IEEE Transactions on Dependable and Secure Computing. 2023;20(6):4733-51. Available from: https://doi.org/10.1109/TDSC.2022.3232813.

[5] Noir Team. Noir;. Accessed: 2026-01-18. Available from: https://noir-lang.org.

[6] ConsenSys. gnark Documentation;. Accessed: 2026-01-18. Available from: https://docs.gnark.consensys.io.

[7] Hochrainer C, Isychev A, Wüstholz V, Christakis M. Fuzzing Processing Pipelines for Zero-Knowledge Circuits. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25). ACM; 2025. p. 783-97. Available from: https://doi.org/10.1145/3719027.3744791.

[8] Xiao D, Liu Z, Peng Y, Wang S. Testing and Exploring Bugs in Zero-Knowledge (ZK) Compilers. In: Network and Distributed System Security Symposium (NDSS). Internet Society; 2025. Accessed: 2026-01-18. Available from: https://doi.org/10.14722/ndss.2025.230530.

[9] Winterer D, Zhang C, Su Z. Validating SMT Solvers via Semantic Fusion. In: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation; 2020. p. 718-30. Available from: https://doi.org/10.1145/3385412.3385985.

[10] Groth J. On the Size of Pairing-Based Non-Interactive Arguments. In: Advances in Cryptology – EUROCRYPT 2016. vol. 9666 of Lecture Notes in Computer Science. Springer; 2016. p. 305-26. Available from: https://doi.org/10.1007/978-3-662-49896-5_11.

[11] Gabizon A, Williamson ZJ, Ciobotaru O. PLONK: Permutations over Lagrange-Bases for Oecumenical Noninteractive Arguments of Knowledge; 2019. Accessed: 2026-01-18. Cryptology ePrint Archive, Report 2019/953. Available from: https://eprint.iacr.org/2019/953.

[12] Ben-Sasson E, Bentov I, Horesh Y, Riabzev M. Scalable, Transparent, and Post-Quantum Secure Computational Integrity; 2018. Accessed: 2026-01-18. Cryptology ePrint Archive, Report 2018/046. Available from: https://eprint.iacr.org/2018/046.

[13] Zcash Foundation. Halo2 Protocol Description;. Accessed: 2026-01-13. https://zcash.github.io/halo2/design/protocol.html.

[14] Ben-Sasson E, Chiesa A, Tromer E, Virza M. Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture. In: 23rd USENIX Security Symposium. USENIX Association; 2014. p. 781-96. Available from: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/ben-sasson.

[15] Chaliasos S, Ernstberger J, Theodore D, Wong D, Jahanara M, Livshits B. SoK: What Don't We Know? Understanding Security Vulnerabilities in SNARKs. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association; 2024. Available from: https://www.usenix.org/conference/usenixsecurity24/presentation/chaliasos.

[16] Veridise. Circom-Pairing: A Million-Dollar ZK Bug Caught Early; 2022. Accessed: 2026-06-05. https://veridise.com/blog/zero-knowledge/circom-pairing-a-million-dollar-zk-bug-caught-early/.

[17] 0xPARC. ZK Bug Tracker; 2024. Software repository. Accessed: 2026-06-05. https://github.com/0xPARC/zk-bug-tracker.

[18] Pailoor S, Chen Y, Wang F, Rodríguez C, Van Geffen J, Morton J, et al. Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs. Proceedings of the ACM on Programming Languages. 2023;7(PLDI):1510-32. Available from: https://doi.org/10.1145/3591282.

[19] Veridise. Picus: Automated Verification of Uniqueness Property for ZKP Circuits;. Software repository. Accessed: 2026-01-18. Available from: https://github.com/Veridise/Picus.

[20] Takahashi H, Kim J, Jana S, Yang J. zkFuzz: Foundation and Framework for Effective Fuzzing of Zero-Knowledge Circuits; 2025. Accessed: 2026-01-18. arXiv preprint arXiv:2504.11961. Available from: https://arxiv.org/abs/2504.11961.

[21] Besier M. Common Circom Pitfalls and How to Dodge Them — Part 1; 2025. Accessed: 2026-01-10. Available from: https://blog.zksecurity.xyz/posts/circom-pitfalls-1/.

[22] Eberhardt J, Tai F. ZoKrates: A Toolbox for zkSNARKs on Ethereum; 2018. Accessed: 2026-01-13. Available from: https://zokrates.github.io/.

[23] ZoKrates Team. ZoKrates GitHub Repository;. Software repository. Accessed: 2026-01-13. Available from: https://github.com/Zokrates/ZoKrates.

[24] Ozdemir A, Brown F, Wahby RS. CirC: Compiler Infrastructure for Proof Systems, Software Verification, and More. In: Proceedings of the 2022 IEEE Symposium on Security and Privacy. IEEE; 2022. p. 2248-66. Available from: https://doi.org/10.1109/SP46214.2022.9833782.

[25] Gupta A, Londhe S, Bisht A, Panda S, Suegami S. ZK Regex. Zenodo; 2025. Software; archived via GitHub repository, original 2022. Available from: https://doi.org/10.5281/zenodo.15603470.

[26] Barr ET, Harman M, McMinn P, Shahbaz M, Yoo S. The Oracle Problem in Software Testing: A Survey. IEEE Transactions on Software Engineering. 2015;41(5):507-25. Available from: https://doi.org/10.1109/TSE.2014.2372785.

[27] Aztec Protocol. Aztec Network;. Accessed: 2026-05-15. https://aztec.network/.

[28] Aztec Protocol. ACIR: Abstract Circuit Intermediate Representation;. Software repository. Accessed: 2026-05-22. https://github.com/noir-lang/acvm.

[29] Mina Protocol. Mina Protocol Documentation;. Accessed: 2026-01-18. Available from: https://docs.minaprotocol.com.

[30] O(1) Labs. o1js: TypeScript Framework for zk-SNARKs and zkApps; 2023. Software repository. Accessed: 2026-01-10. Available from: https://github.com/o1-labs/o1js.

[31] Meckler I, Rao V, Ryan M, Querol A, Spadavecchia J, Wong D, et al.. Introduction to Proof Systems; 2025. Accessed: 2026-01-18. Available from: https://o1-labs.github.io/proof-systems/introduction.html.

[32] StarkWare Industries. The Cairo Book;. Accessed: 2026-01-13. Available from: https://book.cairo-lang.org/.

[33] Goldberg L, Papini S, Riabzev M. Cairo – a Turing-Complete STARK-Friendly CPU Architecture; 2021. Accessed: 2026-01-13. Cryptology ePrint Archive, Paper 2021/1063. Available from: https://eprint.iacr.org/2021/1063.

[34] Aleo Systems. Leo Programming Language;. Accessed: 2026-01-13. Available from: https://developer.aleo.org/leo/.

[35] Aleo Systems. AleoVM Documentation;. Accessed: 2026-01-13. Available from: https://developer.aleo.org/concepts/aleovm/.

[36] Aleo Systems. Varuna: A Universal SNARK for AleoVM;. Software repository. Accessed: 2026-01-13. Available from: https://github.com/AleoHQ/snarkVM.

[37] Miller BP, Fredriksen L, So B. An Empirical Study of the Reliability of UNIX Utilities. Communications of the ACM. 1990;33(12):32-44. Available from: https://doi.org/10.1145/96267.96279.

[38] Manes VJM, Han H, Han C, Cha SK, Egele M, Schwartz EJ, et al. The Art, Science, and Engineering of Fuzzing: A Survey. IEEE Transactions on Software Engineering. 2021;47(11):2312-31. Available from: https://doi.org/10.1109/TSE.2019.2946563.

[39] Godefroid P, Kiezun A, Levin MY. Grammar-Based Whitebox Fuzzing. In: Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM; 2008. p. 206-15. Available from: https://doi.org/10.1145/1375581.1375607.

[40] Yang X, Chen Y, Eide E, Regehr J. Finding and Understanding Bugs in C Compilers. In: Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM; 2011. p. 283-94. Available from: https://doi.org/10.1145/1993498.1993532.

[41] Zalewski M. American Fuzzy Lop;. Software. Accessed: 2026-05-23. https://lcamtuf.coredump.cx/afl/.

[42] McKeeman WM. Differential Testing for Software. Digital Technical Journal. 1998;10(1):100-7.

[43] Chen TY, Cheung SC, Yiu SM. Metamorphic Testing: A New Approach for Generating Next Test Cases. Hong Kong University of Science and Technology; 1998. HKUST-CS98-01.

[44] Zeller A, Hildebrandt R. Simplifying and Isolating Failure-Inducing Input. IEEE Transactions on Software Engineering. 2002;28(2):183-200. Available from: https://doi.org/10.1109/32.988498.

[45] Barrett C, Sebastiani R, Seshia S, Tinelli C. Satisfiability Modulo Theories. In: Biere A, Heule MJH, van Maaren H, Walsh T, editors. Handbook of Satisfiability. vol. 336 of Frontiers in Artificial Intelligence and Applications. IOS Press; 2021. p. 825-85.

[46] de Moura L, Bjørner N. Z3: An Efficient SMT Solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). vol. 4963 of Lecture Notes in Computer Science; 2008. p. 337-40. Available from: https://doi.org/10.1007/978-3-540-78800-3_24.

[47] Barbosa H, Barrett C, Brain M, Kremer G, Lachnitt H, Mann M, et al. cvc5: A Versatile and Industrial-Strength SMT Solver. In: Tools and Algorithms for the Construction and Analysis of Systems. vol. 13243 of Lecture Notes in Computer Science. Springer; 2022. p. 415-42. Available from: https://doi.org/10.1007/978-3-030-99524-9_24.

[48] Park J, Winterer D, Zhang C, Su Z. Generative Type-Aware Mutation for Testing SMT Solvers. Proceedings of the ACM on Programming Languages. 2021;5(OOPSLA):152:1-152:19. Available from: https://doi.org/10.1145/3485529.

[49] Winterer D, Zhang C, Su Z. On the Unusual Effectiveness of Type-Aware Operator Mutations for Testing SMT Solvers. Proceedings of the ACM on Programming Languages. 2020;4(OOPSLA):193:1-193:25. Available from: https://doi.org/10.1145/3428261.

[50] Kremer G, Niemetz A, Preiner M. ddSMT 2.0: Better Delta Debugging for the SMT-LIBv2 Language and Friends. In: Computer Aided Verification. Springer; 2021. p. 231-42. Available from: https://doi.org/10.1007/978-3-030-81688-9_11.

[51] Winterer D, Zhang C, Su Z. Validating SMT Solvers for Correctness and Performance via Grammar-Based Enumeration. Proceedings of the ACM on Programming Languages. 2024;8(OOPSLA2):2378-401. Available from: https://doi.org/10.1145/3689795.

[52] Winterer D, Su Z. ET: A Bounded Exhaustive Testing Tool;. Software repository. Accessed: 2026-06-10. Available from: https://github.com/wintered/ET.

[53] Andoni A, Daniliuc D, Khurshid S, Marinov D. Evaluating the "Small Scope Hypothesis". In: Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '03); 2003. p. 1-12.

[54] Yao P, Huang H, Tang W, Shi Q, Wu R, Zhang C. Fuzzing SMT Solvers via Two-Dimensional Input Space Exploration. In: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21). ACM; 2021. p. 322-35. Available from: https://doi.org/10.1145/3460319.3464803.

[55] Yao P, Huang H, Tang W, Shi Q, Wu R, Zhang C. SMTFuzz;. Software repository. Accessed: 2026-06-10. Available from: https://github.com/rainoftime/smtfuzz.

[56] Xiao D, Liu Z, Peng Y, Wang S. MTZK: Research Artifacts for Testing and Exploring Bugs in Zero-Knowledge Compilers; 2024. Accessed: 2026-04-21. https://sites.google.com/view/mtzk.

[57] Rigorous Software Engineering. circuzz; 2025. Software repository. Accessed: 2026-01-18. Available from: https://github.com/Rigorous-Software-Engineering/circuzz.

[58] ConsenSys. Corset: A Lisp-Based DSL for Zero-Knowledge Constraint Systems;. Software repository. Accessed: 2026-01-15. Available from: https://github.com/Consensys/corset.

[59] iden3. circomlib: Circom library of circuits; 2020. Software repository. Accessed: 2026-04-26. https://github.com/iden3/circomlib.

[60] Hoos HH, Stützle T. SATLIB Benchmark Problems; 2000. Accessed: 2026-05-12. https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html.

[61] Royle G. Minimum Sudoku; 2006. Archived webpage containing the collection of 17-clue Sudoku puzzles. Accessed: 2026-05-12. https://web.archive.org/web/20131019184812/http://school.maths.uwa.edu.au/~gordon/sudoku17.

[62] Ozdemir A, Kremer G, Tinelli C, Barrett C. Satisfiability Modulo Finite Fields. In: Computer Aided Verification. Springer; 2023. p. 163-86. Available from: https://doi.org/10.1007/978-3-031-37703-7_8.

[63] Grassi L, Khovratovich D, Rechberger C, Roy A, Schofnegger M. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association; 2021. p. 519-35. Available from: https://eprint.iacr.org/2019/458.

[64] Gario M, Micheli A. pySMT: A Solver-Agnostic Library for Fast Prototyping of SMT-Based Algorithms. In: Proceedings of the SMT Workshop 2015. vol. 1512. CEUR Workshop Proceedings; 2015. Available from: https://ceur-ws.org/Vol-1512/paper-08.pdf.

[65] Lattner C, Adve V. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. In: Proceedings of the 2nd International Symposium on Code Generation and Optimization (CGO). IEEE; 2004. p. 75-86. Available from: https://doi.org/10.1109/CGO.2004.1281665.

[66] Wieser W. libfaketime: Report Faked System Time to Programs;. Software repository. Accessed: 2026-05-14. https://github.com/wolfcw/libfaketime.

Declarations

Use of Generative AI

I acknowledge the use of Claude Code (Anthropic, `https://claude.ai/code`) and Codex (OpenAI, `https://openai.com/codex`) as coding assistance tools during the implementation of this project. These tools were used for code editing, debugging, and learning about relevant technologies, including ZKP DSLs, Podman, and SMT solvers. I also acknowledge the use of ChatGPT (OpenAI, `https://chatgpt.com`) to support report writing, including grammar correction, phrasing suggestions, and LaTeX formatting. In addition, I used a LLM in the historical bug analysis in Section 5.2 to assist with the classification of scraped GitHub issues. The resulting classifications were used to support the analysis, although I did not manually review every individual classification by hand. All AI-generated code and writing suggestions were reviewed, edited, and verified by me before inclusion. I confirm that the final report, implementation, analysis, and conclusions are my own work.

Ethical Considerations

This project did not involve human participants, personal data, user studies, or the collection of sensitive information, so no direct ethical risks of this kind were identified. The main ethical consideration was that automated compiler testing may uncover security-relevant bugs, since incorrect constraint generation in zero-knowledge proof compilers could have implications for real proof systems. For this reason, any bugs or suspicious behaviours found during the project were treated responsibly, with the aim of producing clear and useful reports for maintainers rather than reporting issues in a way that could increase the risk of misuse.

Availability of Data and Materials

The source code, benchmarks, scripts, and materials used throughout development are available in the main project repository:

`https://github.com/jCabala/zkp-compiler-testing`

The late-stage Noir-focused tool implementation discussed in this report is available separately at:

`https://github.com/jCabala/noir-comp-fuzz`

Together, these repositories contain the testing framework implementation, generated benchmark examples, and scripts required to reproduce the main experimental results. No private user data or sensitive datasets were used in this project.

Appendix A

Circom Architecture

This appendix provides a high-level overview of the Circom compiler architecture and its constraint generation and optimisation pipeline, focusing on the components involved in R1CS generation and constraint simplification. It also briefly analyses the current compiler test-suite coverage.

A.1 Codebase Overview

A.1.1 Directly Relevant to R1CS Generation

Module	Description
constraint_generation	Takes the analyzed Circom program as input and executes it into instantiated compiler constraints and circuit state, with the goal of turning templates, signals, and assignments into a concrete constraint system.
constraint_list	Takes the DAG-mapped, flattened constraints as input and produces a simplified flat constraint list, with the goal of rewriting substitutions and reducing the final system before export. Most of the actual constraint optimization happens here.
circom_algebra	Symbolic math layer used by later stages for expressions, substitutions, constraints, and modular arithmetic. It provides the main simplification helpers used by `constraint_list`.
dag	Takes the instantiated circuit produced by constraint generation and represents it as a structured component graph, with the goal of preserving connectivity for witness bookkeeping, export, and later mapping into a flat constraint list.
constant_tracking	Tiny constant interner used on the constraint path: it stores each distinct constant once, assigns it a stable `CID`, and lets later stages map both from constant value to ID and from ID back to the constant. It is used to build a constant table so later stages can refer to constants by a stable small integer instead of repeating the full value everywhere.

A.1.2 Other

Module	Description
constraint_writers	Writers for R1CS, sym, JSON, logs, and related exported artifacts.
parser	Circom source parsing, include handling, and initial program construction.
type_analysis	Structural and semantic checks over signals, components, buses, dimensions, and scopes.
program_structure	Shared ASTs, diagnostics, symbol metadata, and common compiler data structures.
circom (frontend)	Command-line frontend and top-level compiler orchestration.
compiler (all)	Compiler middle-end for IR lowering, circuit design, and translation before backends.
code_producers	Backend C++ and WebAssembly witness code generation.

A.2 Compiler Architecture

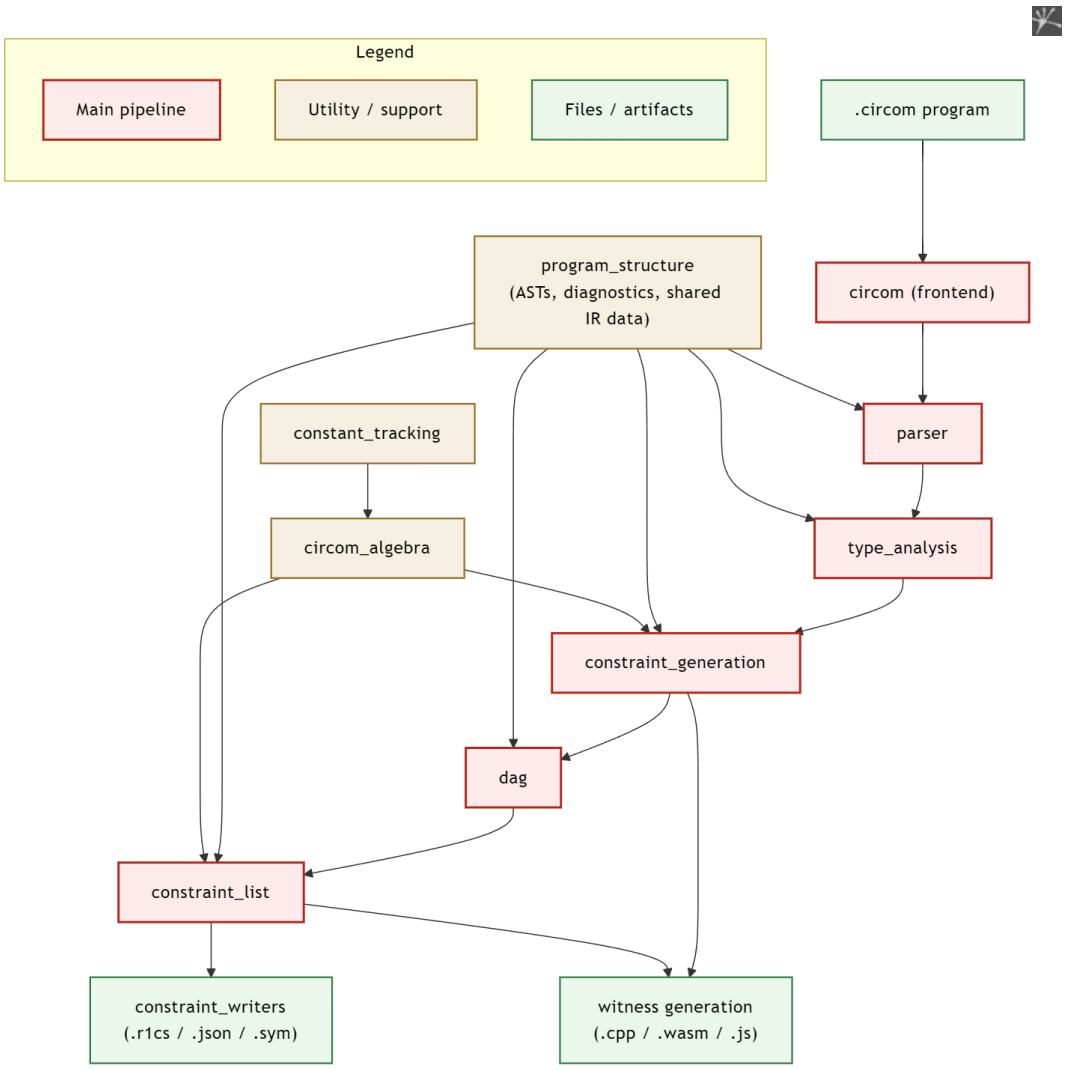

Figure A.1: Circom compiler architecture

Figure A.1 summarises the main compiler modules and their relationships. At a high level, `circom` orchestrates the full compilation pipeline. `parser` and `type_analysis` construct and validate the

program representation, after which `constraint_generation` instantiates the circuit while relying on `circom_algebra` and `constant_tracking` for symbolic manipulation. The resulting circuit is processed by `dag` and `constraint_list`, then exported through `constraint_writers`. The same pipeline also feeds witness-generation backends such as `code_producers`.

A.3 Constraint Simplification Pipeline

Figure A.2 shows the main stages of Circom's constraint simplification pipeline, from simple equality elimination through linear simplification, non-linear propagation, and final witness rebuilding.

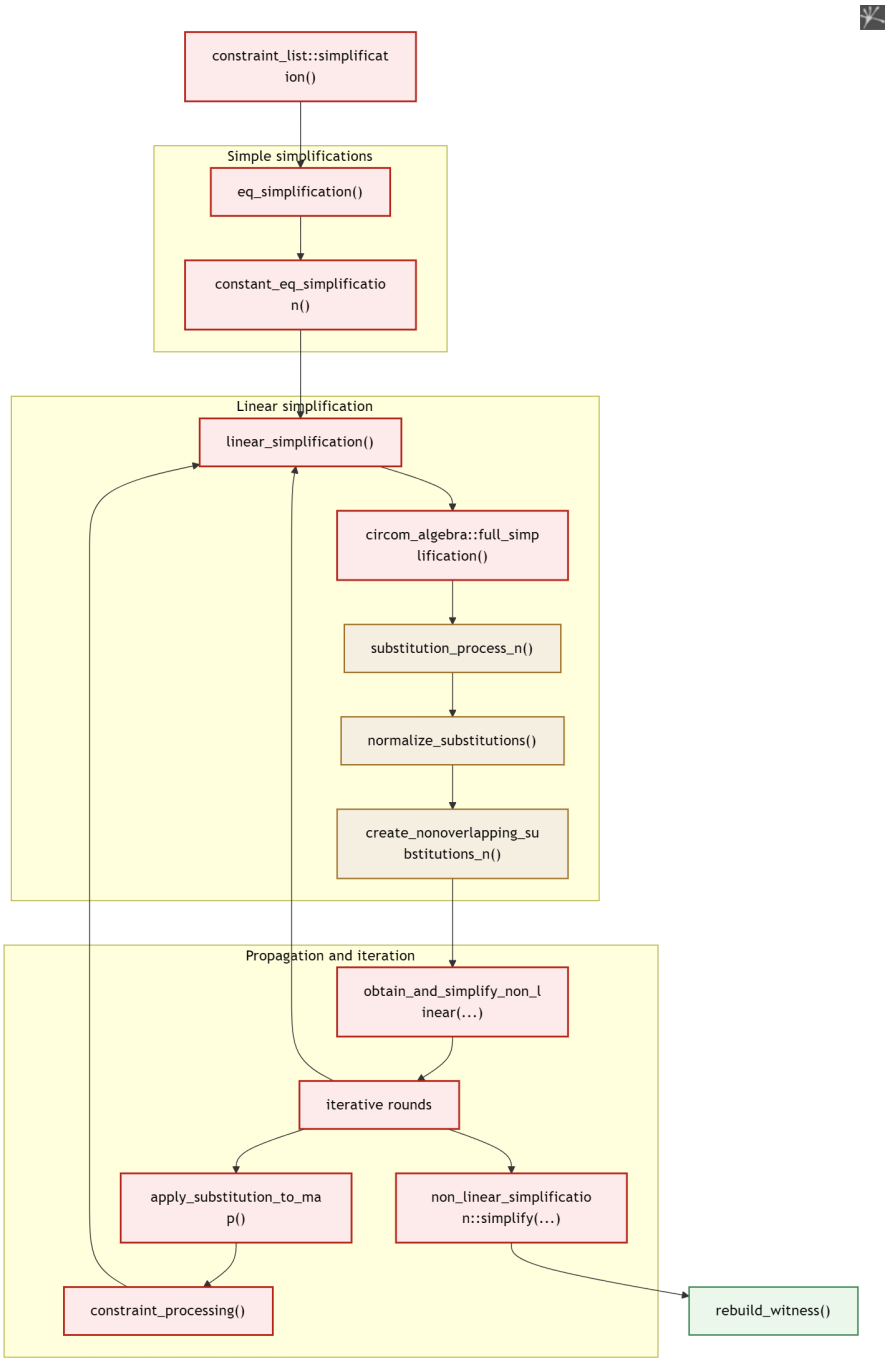

Figure A.2: Constraint simplification pipeline

- **Simple simplifications**: `eq_simplification()` handles pure `signal = signal` equalities, and

`constant_eq_simplification()` handles `signal = constant` equalities.

- **Linear simplification**: `linear_simplification()` groups connected linear constraints into clusters and sends each cluster into `circom_algebra::full_simplification()`. This is the main and most interesting part of the pipeline, where the non-trivial simplifications happen.
- **Propagation and iteration**: Here we simplify the quadratic constraints by substituting the linear constraints identified in the previous section and then restart the entire simplification process on new constraints. `obtain_and_simplify_non_linear(...)` does the first full sweep over the non-linear constraints and tags or indexes them for later updates.
- **Final cleanup**: `non_linear_simplification::simplify(...)` does the last non-linear cleanup, and `rebuild_witness()` repacks signal IDs after deleted and unused signals have been removed.

A.4 Linear Simplification Details

`linear_simplification()` works on clusters of connected linear constraints. For each cluster it calls `circom_algebra::full_simplification()`, which runs:

A.4.1 `substitution_process_n()`

At the simplest level, `substitution_process_n()` does this: look at a linear constraint, choose one signal in it to eliminate, solve the constraint for that signal, record the resulting substitution, and if that clashes with an existing substitution for the same signal, merge the two definitions back into a new linear constraint and keep going.

All `substitution_process_n()` variants do the same high-level job: from a linear constraint with several eliminable signals, choose which signal to solve for and turn that choice into a substitution. The important differences are in the selection rule, in whether normalization happens immediately or is postponed, and in whether the process looks only at the current constraint or at the whole cluster.

A.4.2 Post-processing helpers

`normalize_substitutions()`

`normalize_substitutions()` rescales the collected substitutions into normalized form using batch modular inversion.

`create_nonoverlapping_substitutions_n()`

These passes rewrite substitutions so their right-hand sides no longer depend on other eliminated signals.

A.5 Test-Suite Coverage

The current `test-suite` run covers `1,946` lines out of `29,702` executable lines (`6.6%`). Coverage is concentrated primarily in `code_producers`, which accounts for `1,466` covered lines (`49.3%` coverage), followed by `circom_algebra` with `371` covered lines (`19.4%`), while `program_structure` receives only `109` covered lines (`3.7%`).

Appendix B

Uniqueness Preservation for One-Sided Fusion

This appendix gives a simple argument for why the Picus fusion construction preserves uniqueness under suitable assumptions.

Let $\varphi_1(x, u)$ be a formula over variables x and u, and let $\varphi_2(y, v)$ be a formula over variables y and v. Assume that both formulas have exactly one satisfying assignment. That is, there exists a unique pair (x^*, u^*) satisfying φ_1, and a unique pair (y^*, v^*) satisfying φ_2.

Now introduce a fresh fusion variable z. The fused formula is defined as

$$\Phi(z, y, u, v) = \varphi_1(\mathsf{inv}_x(z, y), u) \wedge \varphi_2(y, v).$$

Here, inv_x is the inverse expression used to reconstruct x from the fusion variable z and the variable y. For example, in the Boolean XOR case,

$$z = x \oplus y \qquad \Rightarrow \qquad x = z \oplus y.$$

Assume that for every fixed value of y, the mapping $z \mapsto \mathsf{inv}_x(z, y)$ is bijective. We now show that Φ has exactly one satisfying assignment.

Since $\varphi_2(y, v)$ has exactly one satisfying assignment, any model of Φ must use the unique values (y^*, v^*). After fixing $y = y^*$, the remaining formula becomes

$$\varphi_1(\mathsf{inv}_x(z, y^*), u).$$

Because $\varphi_1(x, u)$ has exactly one satisfying assignment, we must have

$$\mathsf{inv}_x(z, y^*) = x^* \qquad \text{and} \qquad u = u^*.$$

Since the map $z \mapsto \mathsf{inv}_x(z, y^*)$ is bijective, there exists exactly one value z^* such that

$$\mathsf{inv}_x(z^*, y^*) = x^*.$$

Therefore every satisfying assignment of Φ must be exactly

$$(z^*, y^*, u^*, v^*).$$

Thus, Φ has exactly one satisfying assignment. Hence, one-sided all-occurrence fusion preserves uniqueness under the assumptions above.

Appendix C

Example ET Grammar for Finite-Field SMT Benchmarks

This appendix presents a compact grammar used to generate ET benchmarks over the QF_FF theory. Let

$$p = 21888242871839275222246405745257275088548364400416034343698204186575808495617$$

denote the BN254 field modulus. Then the gramar is given by the following .g4 file:

```
grammar FiniteFieldsCircom;
LP : '(' ; RP : ')' ;
var : 'a' | 'b' ;
ff_const
  : '(as ff1 (_ FiniteField p))'
  | '(as ff(p-1) (_ FiniteField p))'
  | '(as ff((p-1)/2) (_ FiniteField p))' ;
ff_term
  : ff_const
  | var
  | LP ('ff.neg' ff_term
       | 'ff.add' ff_term ff_term
       | 'ff.mul' ff_term ff_term) RP ;
bool_term
  : LP ('not' bool_term
       | 'and' bool_term bool_term
       | 'or' bool_term bool_term
       | 'xor' bool_term bool_term
       | '=' bool_term bool_term
       | 'distinct' bool_term bool_term
       | 'ite' bool_term bool_term bool_term
       | '=' ff_term ff_term
       | 'distinct' ff_term ff_term) RP ;
start
  : LP 'declare-const' var '(_ FiniteField p)' RP
    LP 'declare-const' var '(_ FiniteField p)' RP
    LP 'assert' bool_term RP
    LP 'check-sat' RP
    EOF ;
```

Appendix D

Differential Coverage Analysis of Boolean-only Benchmarks

This appendix describes in detail a differential coverage report I conducted during evaluation, which was previously summarized in Section 5.1.1.

D.1 Differential Coverage: picus-fusion vs circuzz-arithmetic

D.1.1 Summary

circuzz covers 16,443 lines vs SMT-based approach 9,192 lines of the Circom compiler source. Most of the `+7,251` line gap is not directly relevant to R1CS generation: it comes from `circuzz` continuing into witness generation and `.sym` generation, while SMT-based approach stops after the R1CS-facing part of the pipeline.

Neither workload appears to trigger `substitution_process_4()` , so both stay on the older `process_3` path; this suggests the relevant linear-simplification clusters are too small to cross the `process_4` threshold. Additionally, `circuzz` appears to have triggered more than one round of optimisations while the bool-based SMT experiment struggled. That is most likely a problem with the size of the benchmarks. Other than that, there is no evidence that `picus-fusion` misses a distinct core optimisation pass; the remaining directly relevant gap is largely explained by `circuzz-arithmetic` using arithmetic `assert`-driven workloads, while `picus-fusion` is effectively boolean-only.

Module	Circuzz	Picus	Delta	Gap	Relevant
`constraint_generation`	2636	2236	+400	15.2%	Yes
`constraint_list`	819	670	+149	18.2%	Yes
`circom_algebra`	1183	791	+392	33.1%	Yes
`dag`	555	402	+153	27.6%	Yes
`constant_tracking`	21	18	+3	14.3%	Yes
`constraint_writers`	294	251	+43	14.6%	No
`parser`	536	529	+7	1.3%	No
`type_analysis`	1750	1623	+127	7.3%	No
`program_structure`	1565	1428	+137	8.8%	No
`circom` frontend	661	533	+128	19.4%	No
`compiler` all	4058	711	+3347	82.5%	No
`code_producers`	2355	0	+2355	100%	No

Table D.1: Module-level coverage differences between `circuzz-arithmetic` and `picus-fusion`. The *Relevant* column marks modules directly related to constraint generation and optimisation.

D.2 Constraint Generation and Optimisation

D.2.1 Constraint Generation - execute.rs

File	Circuzz	Picus	Delta
`execute.rs`	1283	1010	+273

Both cover: `constraint_execution()`, `execute_statement()`, `execute_expression()`, `perform_assign()`, and the main `add_constraint()` path through <== and ===.

Circuzz only covers: the extra 272 lines in `execute.rs` are concentrated around `preinitialize_component()` / `initialize_component()` / `execute_template_call_complete()`, tag propagation on assignments, inspect-mode handling, and `LogCall` execution. This is largely because Circuzz generated more structurally complex programs involving sub-components and extensive logging throughout execution.

SMT-based approach only covers: 1 isolated line in `execute.rs`.

Neither covers: `constraint_generation` still has `3,584` executable lines uncovered by both tools.

- `execute.rs` (`2,111` lines): mostly `bus` execution, `bus` declarations, `bus` calls, nested `bus` access, and tail diagnostics.
- `component_representation.rs` (`433`): mostly subcomponent inputs and outputs that are buses, tag checks on component I/O, deferred assignments before full component initialization, and bus-field assignment helpers.
- `bus_representation.rs` (`287`): mostly recursive nested-bus initialization, field access, field assignment, and whole-bus assignment.
- `assignment_utils.rs` (`192`): mostly tag propagation and conditional-assignment merge logic.
- `executed_template.rs` (`146`): mostly bus connections, tag-signal recording, ordered bus-symbol generation, and mixed component-array bookkeeping.

This shared gap comes from the current workloads not using buses or tags. A `bus` is a typed bundle of related signals, and a tag is metadata attached to a signal or bus field in its declaration. Tags act as compatibility checks: if a component input expects a tag and the connected signal does not have it, the compiler reports an error. In `.circom`, these features look like:

```
// Tags
signal input {binary} in[8];
signal output {maxbit} out;
// Buses
bus Point() {
    signal x;
    signal y;
}
signal input Point() p;
```

D.2.2 Constraint List / R1CS Optimisation

File	Circuzz	Picus	Delta
`constraint_simplification.rs`	578	465	+113

The high-level simplification pipeline is described in Appenix A.

Both cover: the core simplification loop, `eq_simplification()`, `constant_eq_simplification()`,

the iterative rounds that apply substitutions to remaining constraints, and a large fraction of `linear_simplification()`.

circuzz only covers: the main gap is in the later code that consumes simplification output, especially `apply_substitution_to_map()` and its inner `constraint_processing()` routine. This post-simplification propagation and cleanup path accounts for most of the `578` vs `465` difference in `constraint_simplification.rs`. **That means that the circuits in the SMT-based approach experiment are small enough so they trigger only a single round of optimisations (i.e no new nonlinear constraints are introduced aftert the first round of optimisation so it stops).**

Neither covers: `constraint_list` has `99` executable lines uncovered by both tools.

In `r1cs_porting.rs` (`58` lines), the shared miss is entirely the custom-gate export path: writing the extra `custom_gates_used` and `custom_gates_applied` R1CS sections, collecting custom-gate names and parameters, and walking the DAG to map each application to witness indices. Custom gates are special named gates exported as metadata instead of being flattened into ordinary R1CS constraints, and the official Circom documentation still says that no custom gates are implemented in the `snarkjs` backend yet.

In `constraint_simplification.rs` (`30` lines), the shared miss breaks down into:

- substitution-JSON logging setup and teardown
- special cases for substituting forbidden signals; forbidden signals are signals the simplifier is not allowed to eliminate, and in this pipeline they are exactly signal `0` (the constant-1 signal), main public inputs, main outputs, and custom-gate signals, so these branches rewrite the non-forbidden signal to the forbidden one, not the other way around, and in larger equality clusters keep residual equality constraints between the signals that must remain
- mostly defensive or dead branches: the `build_relevant_set()` non-rename fallback, two `fix_constraint()` calls after fast substitution into equality and accumulated constraint lists, the unreachable `remove_unused = false` branches, and the tiny late-cleanup path that records erased signals from the final non-linear simplifier

D.2.3 Circom Algebra

File	Circuzz	Picus	Delta
`algebra.rs`	820	555	+265
`modular_arithmetic.rs`	147	38	+109
`simplification_utils.rs`	159	141	+18

algebra.rs

The core R1CS constraint types live here: `ArithmeticExpression`, `Constraint`, `Substitution`.

Both cover: `transform_expression_to_constraint_form()`, `Constraint::apply_substitution()`, `clear_signal_from_linear`, `is_linear()`, `is_equality()`, `is_constant_equality()`, and the basic arithmetic operators `add`, `sub`, and `mul`.

circuzz only covers:

- **More ArithmeticExpression operations** (~100 lines): circuzz exercises `div`, `intdiv`, `mod`, `pow`, `shift_l`, `shift_r`, `bit_and`, `bit_or`, `bit_xor`, and comparison operators, while picus-fusion mostly stays in `add`, `sub`, and `mul`. This likely reflects circuzz using more advanced arithmetic inside `assert`-style checks rather than only in expressions that become R1CS.
- **Broader Quadratic{a,b,c} handling** (~60 lines): both tools exercise quadratic constraints, but circuzz reaches more of the code that manipulates a quadratic expression after it is formed, especially adding constants, adding signals, adding linear terms, and multiplying an existing quadratic by a constant. `picus-fusion` still hits quadratic constraints, but they use only

predictable, boolean-style patterns.

- **Constraint manipulation helpers**: `fix_constraint()`, `remove_zero_value_coefficients()`, and substitution decomposition/reconstruction are used during substitution conflict resolution, substitution propagation into existing constraints, and post-processing that makes substitutions non-overlapping. **This is another gap consistent with `picus-fusion` triggering less of the linear-constraint optimization pipeline.**
- **Hash/Eq implementations and serialisation** (~55 lines): used by downstream code emission.

Neither covers: `algebra.rs` still has `258` executable lines uncovered by both tools.

- arithmetic operator edge cases: constant-only and fallback-`NonQuadratic` branches, plus some quadratic special cases
- substitution and constraint plumbing: substitution construction/decomposition, substitution application, remapping, and cleanup helpers
- utility / dead code: display and inspection helpers, witness/export remapping helpers, and the dead `normalize()` stub

D.2.4 DAG

Module	Circuzz	Picus	Delta
`dag`	555	402	+153

The `dag` folder sits between constraint generation and the flattened `constraint_list` representation. It keeps the instantiated circuit in a structured component graph so the compiler can preserve component connectivity, build witness metadata, and then map that graph into the flat constraint form used by simplification.

Both cover: the main DAG-to-constraint-list mapping path, especially `dag/src/lib.rs` and `dag/src/map_to_constraint_list.rs`.

circuzz only covers: the raw DAG `.sym` export path, the DAG witness-generation path, and the direct DAG-side R1CS and JSON export helpers.

Neither covers: `dag` has `232` executable lines uncovered by both tools.

The uncovered lines are mainly in:

- `constraint_correctness_analysis.rs` (`105`)
- `lib.rs` (`74`)
- `r1cs_porting.rs` (`52`)

In `constraint_correctness_analysis.rs`, the uncovered lines are the inspect-only warning-analysis path: constructing `UnconstrainedSignal / UnconstrainedIOSignal` reports, grouping unconstrained examples, `visit_node()`, and `analyse()`.

In `r1cs_porting.rs`, the uncovered lines are entirely the raw-DAG custom-gate R1CS export path.

In `lib.rs`, the uncovered lines are the supporting wrapper and accessor code around those paths: edge/node metadata accessors, the `add_edge()` failure branch, `constraint_analysis()`, raw-DAG export/witness wrapper methods, and the no-main fallback returns of the public/private input/output counters.

D.2.5 Constant Tracking

This is a very small support crate used during constraint generation to intern constants and assign stable IDs to them. The difference is negligible (`21` covered lines for circuzz versus `18` for picus-fusion), but it

Module	Circuzz	Picus	Delta
constant_tracking	21	18	+3

is directly relevant because it sits on the constraint-building path rather than the later witness-generation path.

Both cover: the main constant interning and lookup path.

circuzz only covers: 3 extra lines in constant_tracking/src/lib.rs; the delta is negligible.

Neither covers: just 1 executable line, in the small lookup helper.

D.3 Other Parts of the Compiler (not strictly R1CS generation)

In the picus-fusion run, the witness generation pipeline is not entered. In compilation_user.rs, that code is gated behind `if config.c_flag || config.wat_flag || config.wasm_flag`, so **none of the code below is ever reached by picus-fusion**. This is entirely expected - picus-fusion only needs the R1CS constraint system, not witness generators.

D.3.1 Code Producers

Module	Circuzz	Picus	Delta
code_producers	2355	0	+2355

circuzz only covers: the entire WASM and C++ code emission pipeline: wasm_code_generator.rs (+1,329), c_code_generator.rs (+539), wasm_elements/mod.rs (+347), c_elements/mod.rs (+135), components/mod.rs (+5).

D.3.2 Compiler IR and Circuit Design

Module	Circuzz	Picus	Delta
compiler/	4058	711	+3347

Both cover: the HIR phases that run before constraint generation.

circuzz only covers: translate.rs (+1,023), which converts the AST into the witness-generation IR, plus the circuit_design/ files (build.rs +321, circuit.rs +243, template.rs +253) and ir_processing/ (build_stack.rs +123, reduce_stack.rs +112, build_inputs_info.rs +120, set_arena_size.rs +75). These are all witness-generation-only and do not produce R1CS constraints.

Neither covers: no additional shared gap is discussed here because picus-fusion stops before witness-generation IR construction.

D.3.3 Constraint Writers

Both cover: constraint_writers/src/r1cs_writer.rs; both tools write R1CS output and are nearly identical here (214 covered lines for circuzz, 212 for picus-fusion).

circuzz only covers: json_writer.rs (+26) and log_writer.rs (+9), which picus-fusion does not use.

Neither covers: no additional shared gap is highlighted here; the main writer path is already shared.

Module	Circuzz	Picus	Delta
constraint_writers	294	251	+43

Module	Circuzz	Picus	Delta
program_structure	1565	1428	+137

D.3.4 Program Structure

Both cover: the shared AST types, environment utilities, and memory-slice operations that both frontends rely on.

circuzz only covers: the gap is spread thinly across many files, mainly `error_definition.rs` (+77), `memory_slice.rs` (+32), and `environment.rs` (+1). This mostly reflects circuzz triggering more error and utility paths.

Neither covers: no specific shared blind spot is highlighted here; this section is mainly a thinly spread differential gap.

D.3.5 Frontend / circom

Module	Circuzz	Picus	Delta
circom (frontend)	661	533	+128

Both cover: `main.rs` and the common frontend orchestration path.

circuzz only covers: most of the gap is `compilation_user.rs` (+75), because circuzz enters the compilation branch for witness generation while picus-fusion skips it. `input_user.rs` (+40) reflects circuzz exercising more CLI flag combinations.

picus-fusion only covers: 1 isolated line in `input_user.rs`; there is no broader picus-fusion-only frontend pattern.

Neither covers: no specific shared blind spot is highlighted here; the main difference is whether the compilation branch is entered.

D.3.6 HIR/Frontend

Both cover: the HIR desugaring phases.

circuzz only covers: `merger.rs` (+32), `sugar_cleaner.rs` (+33), and `component_preprocess.rs` (+6). circuzz generates slightly more diverse syntactic constructs, including ternary operators, multi-dimensional arrays, and anonymous components, which exercise more merge and desugar paths.

Neither covers: no specific shared blind spot is highlighted here; the difference is in how much syntactic variety the workloads generate.

D.3.7 Type Analysis

Module	Circuzz	Picus	Delta
type_analysis	1750	1623	+127

Both cover: the main structural and phase-checking path: `type_check.rs`, `unknown_known_analysis.rs`, `signal_declaration_analysis.rs`, and the overall `check_types.rs` pipeline.

circuzz only covers: additional paths in `type_check.rs` (+45), `unknown_known_analysis.rs` (+65), and `symbol_analysis.rs` (+5), because circuzz generates more diverse syntactic constructs.

Neither covers: `custom_gate_analysis.rs` and `type_given_function.rs`.

Appendix E

Artificial Bug Injection - Full Experimental Results

This appendix contains the full per-instance results for the artificial bug injection experiments discussed in Section 5.1.2. Each configuration was executed using multiple independent fuzzing instances with a 5-minute timeout budget. Each instance contains exactly one modified constraint.

E.0.1 SMT - Boolean SAT

Instance	Found	Programs	Time (s)
0	yes	4	55.8
1	yes	4	59.0
2	yes	6	43.7
3	yes	4	85.3
4	yes	4	57.4

Table E.1: SMT Boolean SAT results.

E.0.2 SMT - Boolean UNSAT

Instance	Found	Programs	Time (s)
0	no	12	537.0
1	no	11	512.0
2	no	12	567.2
3	no	7	521.7
4	no	16	507.1

Table E.2: SMT Boolean UNSAT results.

E.0.3 SMT - Boolean Picus

Instance	Found	Programs	Time (s)
0	yes	4	52.4
1	yes	6	37.1
2	yes	4	61.9
3	yes	4	237.5
4	yes	4	92.0

Table E.3: SMT Boolean Picus results.

E.0.4 CIRCUZZ - Standard

Instance	Found	Programs	Time (s)
0	no	51	317.0
1	no	62	309.5
2	no	52	337.7
3	yes	55	325.3
4	yes	7	90.4

Table E.4: CIRCUZZ Standard results.

E.0.5 CIRCUZZ - Picus

Instance	Found	Programs	Time (s)
0	no	6	325.1
1	no	15	307.0
2	no	0	467.7
3	no	15	308.5
4	no	16	301.5

Table E.5: CIRCUZZ Picus results.

E.0.6 Overall Aggregate

Configuration	Bugs	Programs	Bugs/Prog	Bugs/Sec
SMT SAT	5/5	22	0.2273	0.0166
SMT UNSAT	0/5	58	0.0000	0.0000
SMT Picus	5/5	22	0.2273	0.0104
Circuzz Std.	2/5	227	0.0088	0.0014
Circuzz Picus	0/5	52	0.0000	0.0000

Table E.6: Overall aggregate results.